



# Comparative evaluation of matrix-free implicit viscosity solvers for GPU-accelerated SPH simulation of highly viscous fluids

Hee Sang Yoo<sup>1,2</sup> · Eung Soo Kim<sup>2</sup>

Received: 9 May 2025 / Revised: 21 August 2025 / Accepted: 3 September 2025 / Published online: 29 September 2025  
© The Author(s) 2025

## Abstract

The demand for accurate and efficient simulation of highly viscous fluids is growing across engineering domains such as energy systems, advanced manufacturing, and multi-phase transport. Applications including crude oil pipeline flow, extrusion-based additive manufacturing, and non-Newtonian material modeling frequently involve fluids with extreme viscosity or shear-dependent behavior. In such cases, implicit viscosity solvers are often favored over traditional explicit SPH solvers, which are hindered by stringent time step constraints that lead to high computational costs in large-scale simulations. In this study, we enhance existing matrix-free implicit viscosity solvers and evaluate their effectiveness within a GPU-accelerated smoothed particle hydrodynamics (SPH) framework. To improve computational efficiency under complex boundary conditions, we reformulate the coefficient matrix to satisfy symmetry, enabling more efficient implicit computation. A wide range of benchmark cases is considered, including steady and transient flows, single- and multi-phase Newtonian fluids, and diverse non-Newtonian rheologies. We also compare solver performances such as convergence behavior, accuracy, and computing time under multiple boundary condition implementations. The results demonstrate clear trends in solver robustness and efficiency across flow regimes, offering practical insights into the deployment of matrix-free implicit solvers in high-fidelity SPH simulations of highly viscous fluids.

**Keywords** Implicit viscosity solver · Highly viscous · SPH · GPU · Matrix-free

## 1 Introduction

In recent years, the simulation of highly viscous fluids has garnered significant attention within the research community, owing to its wide range of applications across various engineering disciplines [1]. Highly viscous fluids play a crucial role in fields such as crude oil transportation [2, 3] and petroleum product pipeline transit [4], as well as in the manufacturing processes of advanced materials [5–7]. Beyond merely simulating highly viscous fluids, research into these fluids has expanded to include applications in phase change phenomena, natural disaster modeling, physically based graphic animations, multi-physics interactions, and material science experimentation, highlighting its growing

relevance and versatility in addressing complex real-world challenges [1, 8–11].

To capture the complex physical phenomena of highly viscous fluids, numerical methods like the smoothed particle hydrodynamics (SPH), moving particle semi-implicit (MPS), lattice Boltzmann method (LBM), and phase field methods have recently been explored [12–15]. Among them, SPH, a leading mesh-free Lagrangian method, has gained widespread use in engineering simulations due to its natural ability to handle free surface flows and multi-phase interactions without the need for complex interface tracking.

However, simulations involving high viscosity are numerically challenging. Explicit time integration schemes require prohibitively small time steps for numerical stability, particularly for flows with high viscosity. To overcome this limitation, implicit integration of the viscous term has been adopted for various applications using weakly compressible SPH (WCSPH). Incompressible SPH (ISPH), and as well as in similar particle-based methods [1, 16–22]. The use of implicit viscosity solvers was actively introduced in the computer graphics field, where robust and stable simula-

✉ Eung Soo Kim  
kes7741@snu.ac.kr

<sup>1</sup> DES, IRESNE, CEA, Cadarache, Saint-Paul-lès-Durance, France

<sup>2</sup> Department of Nuclear Engineering, Seoul National University, 1 Gwanak-ro, Gwanak-gu, Seoul, South Korea

tion of thick fluids was essential [23–26]. These techniques were later re-examined in engineering contexts for simulating highly viscous flows such as lava flows and non-Newtonian fluids [1, 16, 17].

Among these studies, several have made notable contributions by rigorously addressing the technical aspects of implicit solver implementation. For instance, Krylov subspace methods have been implemented within GPU parallelized framework to solve the large linear systems that arise in semi-implicit WSPH formulations [16, 17]. The conjugate gradient (CG) method has been used to solve the linear system for fluid particles while treating boundary conditions externally [16], whereas BiCGSTAB has been applied to implicitly incorporate both the viscous term and boundary conditions [14]. In the incompressible context, semi-implicit approaches have been further extended to handle highly viscous fluid flows [19, 27]. Slight algorithmic modifications have been employed to enhance stability under extreme viscosity, particularly in cases involving phase change effects [19]. Fully implicit schemes have also been developed, demonstrating robust and stable performance across a wide range of time steps and boundary conditions, for both Newtonian and non-Newtonian fluids [1].

Despite their advantages, implicit solvers require solving large-scale linear systems that are sparse and dynamically structured due to particle-based discretization. Krylov subspace methods such as Conjugate Gradient (CG) and BiCGSTAB are commonly employed due to their superior convergence properties over classical stationary methods like Jacobi or SOR [28, 29]. In SPH, however, each particle typically interacts with many neighbors, resulting in sparse matrices with relatively high fill-in ratios and memory demands [30, 31]. In this context, multiplication between a matrix and vector is implemented in a matrix-free manner, where the effect of the coefficient matrix is computed directly through particle interactions. This approach avoids the need to assemble and store sparse matrices, reduces memory usage, and is well-suited for GPU parallelization due to its localized and data-parallel structure [32, 33].

Building on the matrix-free formulation suited for GPU implementation, we adopt semi-implicit integration of the viscous term within an explicit incompressible SPH framework, which has been widely used in recent studies. We extend prior work by incorporating additional boundary condition treatments and reformulating the coefficient matrix to enforce symmetry. This symmetric reformulation allows the use of more efficient solvers such as CG and simplifies matrix construction by eliminating the need to explicitly distinguish between diagonal and off diagonal terms under complex boundary conditions. We implement several matrix-free iterative solvers, including Jacobi, CG, BiCGSTAB, and their preconditioned variants, and evaluate their performance in terms of convergence, computational efficiency,

and accuracy across Newtonian and non-Newtonian flows. The comparative results highlight the practical benefits of symmetric reformulation and preconditioning strategies for scalable and robust SPH simulations of highly viscous fluids.

This paper is organized into six sections. Section 2 provides an overview of the SPH scheme used in this study. Section 3 details the construction of the implicit viscosity solver within SPH. Section 4 elaborates on the implementation and GPU parallelization of various implicit viscosity solvers in detail. Section 5 presents several numerical benchmarks, comparing the efficiency, accuracy, and convergence of the different viscosity solvers. Finally, Sect. 6 summarizes the paper and offers concluding remarks.

## 2 Discretization of governing equations

In this section, the governing equations for incompressible flow are extended to encompass free surface flows, multi-phase flows, and both Newtonian and non-Newtonian fluid behaviors. To solve these equations, the explicit incompressible smoothed particle hydrodynamics (EISPH) method is employed. The section begins with a brief overview of the SPH discretization, followed by a detailed derivation and discretization process of the EISPH method, which forms the foundation of the computational approach adopted in this study.

### 2.1 Governing equations for incompressible flows

The governing equations for isothermal and incompressible fluid flow are based on the principles of mass conservation and momentum conservation, as follows:

$$\nabla \cdot \mathbf{u} = 0, \quad (1)$$

$$\frac{D\mathbf{u}}{Dt} = -\frac{\nabla P}{\rho} + \frac{1}{\rho} \nabla \cdot \boldsymbol{\tau} + \mathbf{g}, \quad (2)$$

where  $\mathbf{u}$  represents velocity,  $\rho$  is density,  $P$  denotes pressure,  $\boldsymbol{\tau}$  represents the viscous stress tensor, and  $\mathbf{g}$  denotes gravitational acceleration.

The viscous stress tensor  $\boldsymbol{\tau}$  is related to the shear rate tensor  $\dot{\boldsymbol{\gamma}}$  as:

$$\boldsymbol{\tau} = \mu \dot{\boldsymbol{\gamma}} \quad (3)$$

where  $\mu$  is the dynamic viscosity for Newtonian fluids. For non-Newtonian fluids,  $\mu$  is replaced by the apparent dynamic viscosity  $\mu_{\text{eff}}$ , which depends on the local shear rate and can be computed using various non-Newtonian rheological models.

$$\boldsymbol{\tau} = \mu_{\text{eff}} \dot{\boldsymbol{\gamma}} \quad (4)$$

However, for simplicity, the general description of dynamic viscosity is written in  $\mu$  in this study. The shear rate tensor  $\dot{\boldsymbol{\gamma}}$  is defined as:

$$\dot{\boldsymbol{\gamma}} := \nabla \mathbf{u} + \nabla \mathbf{u}^T. \quad (5)$$

with  $\nabla \mathbf{u}$  being the velocity gradient tensor.

## 2.2 Projection-based method

The first-order projection method, which stems from the Helmholtz-Leray decomposition, is widely utilized to solve the governing equations of incompressible flows by ensuring velocity fields satisfy the divergence-free condition while solving the momentum equations [34]. This method involves two steps: initially, the intermediate velocity ( $\mathbf{u}^*$ ) is solved based on forces except for the pressure force. Subsequently, the corrected velocity is corrected by the pressure force.

The prediction step

$$\mathbf{u}^* = \mathbf{u}^n + \Delta t \left( \frac{1}{\rho} \nabla \cdot \boldsymbol{\tau} + \mathbf{g} \right) \quad (6)$$

The correction step

$$\mathbf{u}^{n+1} = \mathbf{u}^* + \Delta t \left( -\frac{\nabla P^{n+1}}{\rho} \right) \quad (7)$$

Between these two steps, it is necessary to compute the pressure field that enforces the incompressibility constraint. From Eq. (7) and enforcement of divergence-free condition ( $\nabla \cdot \mathbf{u}^{n+1} = 0$ ) on velocity field of next time step, the Pressure Poisson Equation (PPE) can be derived as:

$$\nabla \cdot \left( \frac{1}{\rho} \nabla P^{n+1} \right) = \frac{\nabla \cdot \mathbf{u}^*}{\Delta t} \quad (8)$$

## 2.3 SPH approximation

The fundamental approximation of a scalar function  $f(\mathbf{r})$  at a point  $\mathbf{r}$  in SPH is defined as the convolution over a support domain  $\Omega$ . The weighing factor at arbitrary position  $\mathbf{r}'$  is defined using a weighting function  $W(\mathbf{r} - \mathbf{r}', h)$  with a smoothing length  $h$  [35]:

$$f(\mathbf{r}) = \int_{\Omega} f(\mathbf{r}') W(\mathbf{r} - \mathbf{r}', h) d\mathbf{r}'. \quad (9)$$

In discretized particle representations of continuous fluid systems described in Fig. 1, the function  $f(\mathbf{r})$  of the  $i$  th

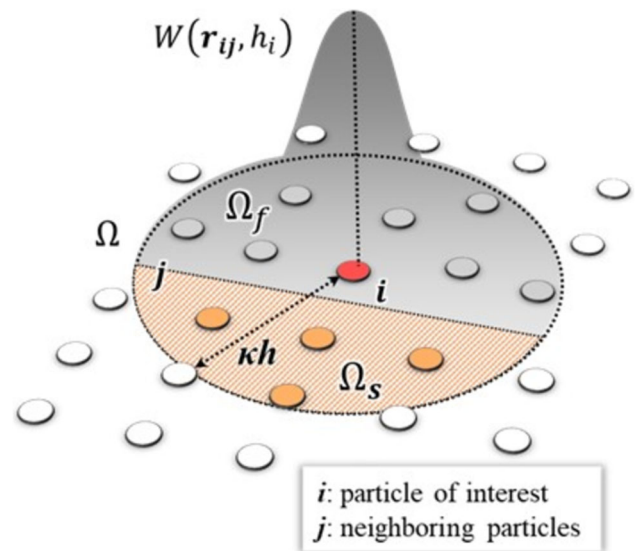


Fig. 1 Schematic view of SPH integration in discretized system

particle is expressed as:

$$f_i := f(\mathbf{r}_i) = \sum_j f_j W_{ij}(|\mathbf{r}_{ij}|, h) V_j. \quad (10)$$

In Fig. 1, the sub-support domain  $\Omega_f$  and  $\Omega_s$  demonstrates the support domain of fluid and solid particles. The subscript  $j$  denotes neighboring particles,  $m$ ,  $\rho$ , and  $V := m/\rho$  represent mass, density, and volume, respectively. The weighting function  $W_{ij}$  with  $\mathbf{r}_{ij} = \mathbf{r}_i - \mathbf{r}_j$  is simplified as  $W_{ij}$ . For this study, the Wendland C6 kernel function is utilized as the weighting function due to its outstanding stability [36]:

$$W(q) = \begin{cases} C(1-q)^8(1+8q+25q^2+32q^3), & \text{if } 0 \leq q < 1 \\ 0, & \text{if } q \geq 1 \end{cases} \quad (11)$$

Here,  $q = r/(\kappa h)$ ,  $C = 78/7\pi(\kappa h)^2$  in 2D, and  $\kappa$  is the scale factor defining the effective area of the kernel smoothing function and set to be 3 in this study. The smoothing length  $h = 1.5\Delta x$ , where  $\Delta x$  is the initial particle spacing. Therefore, the  $i$  th particle interacts with neighboring particles within a support radius of  $\kappa h$ .

The general form of SPH gradient, divergence, and Laplacian operators for a scalar function  $f_i$  and a vector function  $\mathbf{F}_i$  can be derived as follows [37]:

$$\nabla f_i = \sum_j [f_j \pm f_i] \nabla_i W_{ij} V_j, \quad (12)$$

$$\nabla \cdot \mathbf{F}_i = \sum_j [\mathbf{F}_j \pm \mathbf{F}_i] \cdot \nabla_i W_{ij} V_j, \quad (13)$$

$$\nabla^2 f_i = \sum_j 2[f_j - f_i] \frac{\mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{|\mathbf{r}_{ij}|^2} V_j. \quad (14)$$

## 2.4 EISPH time integration

The discrete governing Eqs. (6) to (8) for fluid particles in the previous section can be computed through the following steps [38]:

- (1) Compute acceleration excluding pressure and viscous force  $\mathbf{a}_{\text{remain}}$
- (2) Compute first intermediate velocity  $\mathbf{u}_i^* = \mathbf{u}_i^n + (\mathbf{a}_{vis} + \mathbf{a}_{\text{remain}})\Delta t$
- (3) Compute PPE to get pressure  $P_i^{n+1}$
- (4) Compute pressure force  $\mathbf{a}_{\text{pres}}$
- (5) Compute corrected velocity  $\mathbf{u}_i^{n+1} = \mathbf{u}_i^* + \mathbf{a}_{\text{pres}}\Delta t$
- (6) Compute corrected position  $\mathbf{r}_i^{n+1}$

### 2.4.1 Newtonian fluid

In this study, the discretization of pressure force and viscous terms for Newtonian fluids in both single-phase and multi-phase flows is expressed as follows [39, 40]:

$$\mathbf{a}_{vis} = \frac{1}{\rho_i} \sum_{j=1} \overline{\mu}_{ij} (\mathbf{u}_i^n - \mathbf{u}_j^n) \frac{2\mathbf{r}_{ij} \cdot \nabla_i W_{ij} V_j}{|\mathbf{r}_{ij}|^2 + \eta^2}, \quad (15)$$

$$\mathbf{a}_{\text{pres}} = -\frac{1}{\rho_i} \sum_j 2P_i^{n+1} \nabla_i W_{ij} V_j. \quad (16)$$

The interparticle viscosity  $\overline{\mu}_{ij}$  is defined as the harmonic mean of the  $\mu_i$  and  $\mu_j$ :

$$\overline{\mu}_{ij} := \frac{2\mu_i\mu_j}{\mu_i + \mu_j}, \quad (17)$$

and the interparticle pressure is defined as [41, 42]:

$$\overline{P}_{ij}^{n+1} := \left( \frac{P_i^{n+1}\rho_i + P_j^{n+1}\rho_j}{\rho_i + \rho_j} \right) \quad (18)$$

For multi-phase flows, the PPE is discretized as [43]:

$$\begin{aligned} & \sum_{j=1}^N (P_i^{n+1} - P_j^{n+1}) \frac{2\mathbf{r}_{ij} \cdot \nabla W(\mathbf{r}_{ij}, h) m_j}{|\mathbf{r}_{ij}|^2 + \eta^2} \frac{1}{\rho_j} \\ &= \frac{\sum_j \overline{\rho}_{ij} \mathbf{u}_{ji}^* \cdot \nabla W(\mathbf{r}_{ij}, h) \frac{m_j}{\rho_j}}{\Delta t}. \end{aligned} \quad (19)$$

where, the interparticle density  $\overline{\rho}_{ij}$  is the harmonic mean of the  $\rho_i$  and  $\rho_j$ .

$$\overline{\rho}_{ij} := \frac{2\rho_i\rho_j}{\rho_i + \rho_j}. \quad (20)$$

By employing the first step of the Jacobi method in PPE, an explicit formulation of the PPE in EISPH is written as [38, 43, 44]:

$$\sum_{j=1} (P_i^{n+1} - P_j^n) \frac{2\mathbf{r}_{ij} \cdot \nabla W_{ij} V_j}{|\mathbf{r}_{ij}|^2 + \eta^2} = \frac{\sum_j \overline{\rho}_{ij} \mathbf{u}_{ji}^* \cdot \nabla_i W_{ij} V_j}{\Delta t} \quad (21)$$

### 2.4.2 Non-Newtonian fluid

For Newtonian fluids, the dynamic viscosity remains constant under isothermal conditions. However, for non-Newtonian fluids, the apparent dynamic viscosity varies based on the local velocity distribution, requiring updates to the particle viscosity at each time step before computing the viscous force. Among the various viscous terms utilized modeling non-Newtonian fluids in previous literatures [21, 45–48], the simple viscous term for its simplicity [21, 39, 46]:

$$\mathbf{a}_{vis} = \frac{1}{\rho_i} \sum_{j=1} \overline{\mu}_{ij}^n (\mathbf{u}_i^n - \mathbf{u}_j^n) \frac{2\mathbf{r}_{ij} \cdot \nabla_i W_{ij} V_j}{|\mathbf{r}_{ij}|^2 + \eta^2}, \quad (22)$$

where interparticle viscosity  $\overline{\mu}_{ij}^n$  can be defined as the harmonic mean of the  $\mu_i$  and  $\mu_j$  at  $n$ -th step. The apparent dynamic viscosity of non-Newtonian fluids is typically expressed as a function of the shear rate tensor which is dependent on the flow rheological models.

$$\overline{\mu}_{ij}^n = \mu_{\text{eff}} = f(|\dot{\gamma}|) \quad (23)$$

The scalar magnitude of the shear rate,  $|\dot{\gamma}|$  is given by [39]:

$$|\dot{\gamma}| = \sqrt{\frac{1}{2} \sum_a \sum_b \dot{\gamma}_{ab} \dot{\gamma}_{ba}}. \quad (24)$$

where the subscript  $a$  and  $b$  are the direction  $x$ ,  $y$ , and  $z$ . The shear rate tensor  $\dot{\gamma}_i$  is evaluated around each particle to compute its components:

$$\dot{\gamma}_i := \begin{bmatrix} 2\left(\frac{\partial u}{\partial x}\right)_i & \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}\right)_i \\ \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}\right)_i & 2\left(\frac{\partial v}{\partial y}\right)_i \end{bmatrix} = \begin{bmatrix} 2\sum_j \left[(u_j - u_i) \frac{\partial W_{ij}}{\partial x}\right] V_j & \sum_j \left[(u_j - u_i) \frac{\partial W_{ij}}{\partial y} + (v_j - v_i) \frac{\partial W_{ij}}{\partial x}\right] V_j \\ \sum_j \left[(u_j - u_i) \frac{\partial W_{ij}}{\partial y} + (v_j - v_i) \frac{\partial W_{ij}}{\partial x}\right] V_j & 2\sum_j \left[(v_j - v_i) \frac{\partial W_{ij}}{\partial y}\right] V_j \end{bmatrix}. \quad (25)$$

Therefore, final discretized governing equations in this study are summarized as follows. These general equations are utilized to solve single/multi-phase Newtonian fluid flow and non-Newtonian fluid flow in this study.

The prediction step

$$\mathbf{u}^* = \mathbf{u}^n + \Delta t \left( \frac{1}{\rho_i} \sum_{j=1}^N \overline{\mu_{ij}^n} (\mathbf{u}_i^n - \mathbf{u}_j^n) \frac{2\mathbf{r}_{ij} \cdot \nabla_i W_{ij} V_j}{|\mathbf{r}_{ij}|^2 + \eta^2} + \mathbf{g} \right) \quad (26)$$

PPE

$$\begin{aligned} & \sum_{j=1}^N \left( P_i^{n+1} - P_j^n \right) \frac{2\mathbf{r}_{ij} \cdot \nabla W(\mathbf{r}_{ij}, h) m_j}{|\mathbf{r}_{ij}|^2 + \eta^2} \frac{1}{\rho_j} \\ &= \frac{\sum_j \overline{\rho_{ij}} \mathbf{u}_{ji}^* \cdot \nabla W(\mathbf{r}_{ij}, h) \frac{m_j}{\rho_j}}{\Delta t}. \end{aligned} \quad (27)$$

The correction step

$$\mathbf{u}^{n+1} = \mathbf{u}^* + \Delta t \left( -\frac{1}{\rho_i} \sum_j 2\overline{P^{n+1}_{ij}} \nabla_i W_{ij} V_j \right) \quad (28)$$

### 3 Implicit time integration of viscous term

In this section, we present the discretization and construction of the implicit viscosity solver algorithm, including a detailed explanation of the time integration sequence. We also describe four different velocity boundary conditions and elaborate on how these are integrated into the implicit viscosity solver.

#### 3.1 Implicit time integration of viscous term in EISPH

The transition from the explicit algorithm to the semi-implicit algorithm with implicit integration of viscous forces in the WCSPH framework has been thoroughly explained in previous studies [16, 17]. In the explicit approach, all forces are simultaneously solved, whereas in the implicit approach, the process is divided into two steps. Initially, all forces excluding the viscous force are resolved to ascertain the alteration in velocity. Subsequently, the component of velocity change attributable to viscous forces is implicitly resolved.

The ISPH method employed in this study is already partitioned into two stages, as detailed in the preceding section. It is divided into resolving non-pressure forces and resolving pressure forces, as outlined. Previous studies attempting to handle viscous forces implicitly in ISPH-based or similar MPS methods have employed varying methodologies [1, 19–22, 27].

In this study, the computation of forces is divided into three distinct stages as in similar previous projection-based method [19]. Initially, non-pressure forces, such as pressure and viscous forces, are calculated to predict the velocity field. Following this, the pressure is computed, and the velocity is updated to account for the changes induced by the pressure gradient, resulting in a second intermediate velocity that satisfies the divergence-free condition. Finally, using this second intermediate velocity, an implicit viscosity solver is employed to compute the final corrected velocity. The modified algorithm is structured as follows, as depicted in Fig. 2 alongside the explicit method:

- (1) Compute acceleration excluding pressure and viscous force  $\mathbf{a}_{\text{remain}}$
- (2) Compute first intermediate velocity  $\mathbf{u}_i^* = \mathbf{u}_i^n + \mathbf{a}_{\text{remain}} \Delta t$
- (3) Compute PPE
- (4) Compute pressure force  $\mathbf{a}_{\text{pres}}$
- (5) Compute second intermediate velocity  $\mathbf{u}_i^{**} = \mathbf{u}_i^* + \mathbf{a}_{\text{pres}} \Delta t$
- (6) Compute corrected velocity with implicit viscosity solver (IVS)  $\mathbf{u}_i^{n+1}$
- (7) Compute corrected position  $\mathbf{r}_i^{n+1}$

#### 3.2 Construction of the implicit viscosity solver

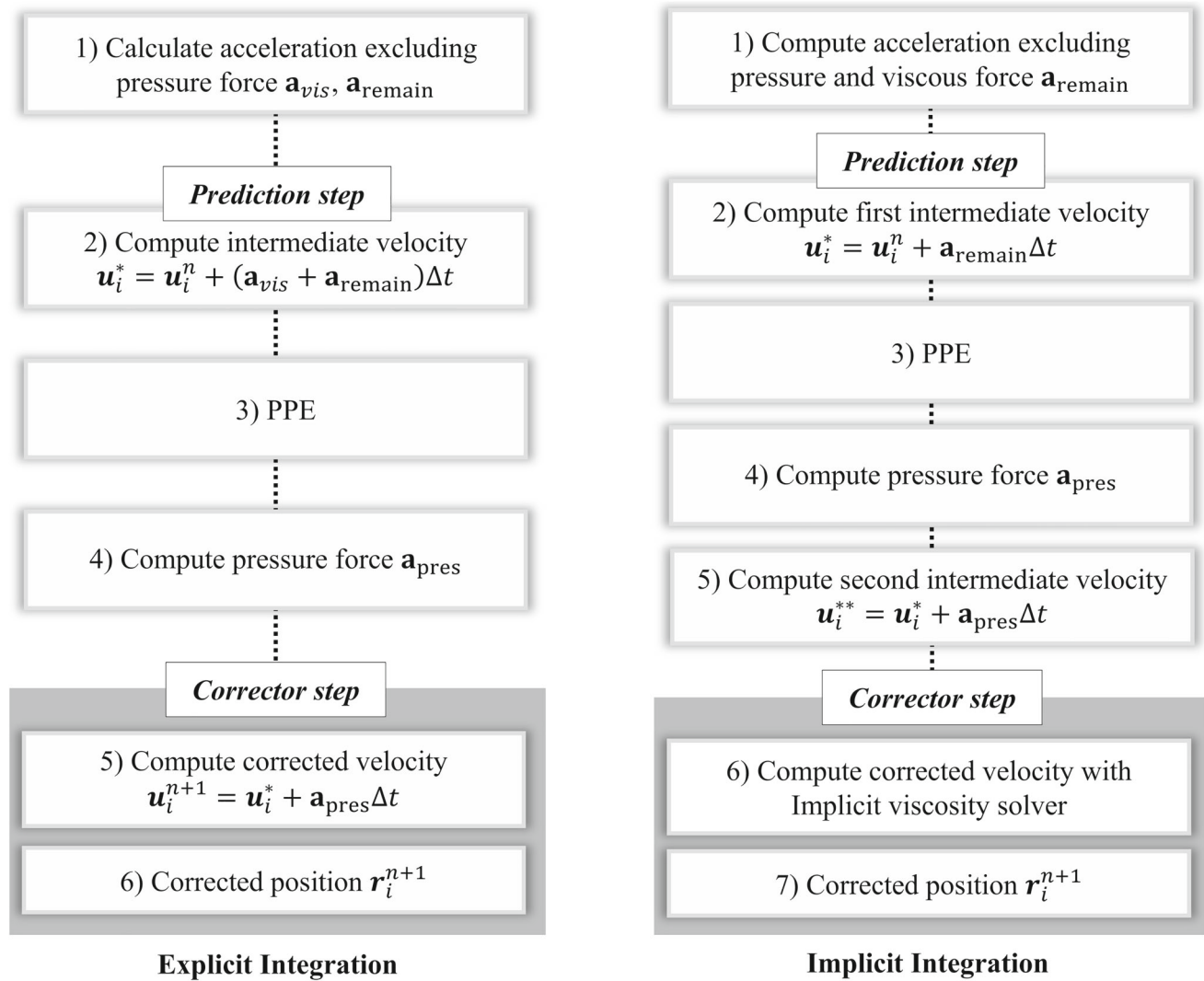
The algorithms described, except for number 6, can be obtained from the preceding sections. To solve viscous force implicitly, the system of linear equations must be defined:

$$\mathbf{A}^n \mathbf{x}^n = \mathbf{b}^n. \quad (29)$$

Here,  $\mathbf{A}^n = [a_{ij}^n]$  is referred to as the coefficient matrix,  $\mathbf{b}^n = [b_i^n]$  is the right-hand side vector and  $\mathbf{x}^n = [x_i^n]$  is the unknown vector, which represents the anticipated velocity ( $\mathbf{u}_i^{**}$ ) and the velocity at the next step ( $\mathbf{u}_i^{n+1}$ ) in this study, respectively. The implicit viscosity solver (number 6) is derived as follows. First, the relationship between the final velocity ( $\mathbf{u}_i^{n+1}$ ) and the second intermediate velocity ( $\mathbf{u}_i^{**}$ ) is as follows:

$$\begin{aligned} \mathbf{u}_i^{n+1} &= \mathbf{u}_i^{**} + \mathbf{a}_{\text{vis}} \Delta t \\ &= \mathbf{u}_i^{**} + \frac{1}{\rho_i} \sum_{j=1}^N \overline{\mu_{ij}^n} (\mathbf{u}_i^{**} - \mathbf{u}_j^{**}) \frac{2\mathbf{r}_{ij} \cdot \nabla_i W_{ij} V_j}{|\mathbf{r}_{ij}|^2 + \eta^2}, \end{aligned} \quad (30)$$

$$K_{ij}^n := \frac{\overline{\mu_{ij}^n}}{\rho_i} \frac{2\mathbf{r}_{ij} \cdot \nabla_i W_{ij} V_j}{|\mathbf{r}_{ij}|^2 + \eta^2}. \quad (31)$$



**Fig. 2** Computing algorithm of projection-based ISPH method with explicit and implicit time integration of viscous forces

For the implicit integration of the viscous force term, it can be rewritten as:

$$\mathbf{u}_i^{n+1} = \mathbf{u}_i^{**} + \mathbf{a}_{vis}\Delta t = \mathbf{u}_i^{**} + \sum_{j=1}^N K_{ij}^n (\mathbf{u}_i^{n+1} - \mathbf{u}_j^{n+1})\Delta t. \quad (32)$$

Rearranging the terms with  $\mathbf{u}_i^{n+1}$  on the left side, the equation takes on a set of N linear equations where N is number of particles:

$$\mathbf{u}_i^{n+1} \left[ 1 - \sum_{j=1}^N K_{ij}^n \Delta t \right] + \sum_{j=1}^N K_{ij}^n \mathbf{u}_j^{n+1} \Delta t = \mathbf{u}_i^{**}. \quad (33)$$

This equation can be rewritten in matrix formulation:

$$\mathbf{A}^n \mathbf{u}_i^{n+1} = \mathbf{u}_i^{**}. \quad (34)$$

$$\mathbf{A}^n = [a_{ij}^n] = \begin{cases} \left[ 1 - \sum_j K_{ii}^n \Delta t \right] & i = j (\text{diagonal term}) \\ K_{ij}^n \Delta t & i \neq j, j \in \Omega \\ 0 & i \neq j, j \notin \Omega \end{cases} \quad (35)$$

This matrix exhibits characteristics of a strictly diagonally dominant (SDD) matrix if the smoothing kernel satisfies the monotonic decreasing property [16]. Therefore, it is nonsingular and can be solved using iterative methods.

### 3.3 Boundary condition

To apply no-slip boundary conditions for solid walls, four different methods are considered. These methods differ in how

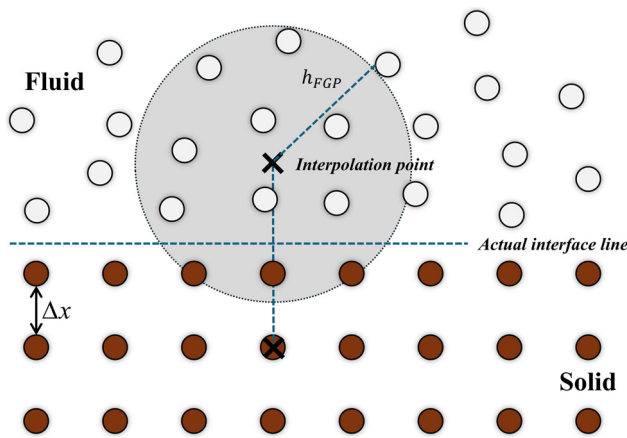


Fig. 3 Concept of fixed ghost particles method

solid particle velocities are defined and how the wall boundary influences are incorporated into the system of equations.

### 3.3.1 Fixed wall particles (FWPs) [1, 40]

The first method is a very simple method to apply the desired velocity  $U_{wall}$  to solid particle. In this study, the desired velocity is set as zero.

$$\mathbf{u}_{i,solid} = U_{wall}. \quad (36)$$

### 3.3.2 Fixed ghost particles (FGPs) [49]

The second approach, referred as the fixed ghost particles (FGP) method, involves the generation of virtual ghost nodes distributed symmetrically within the fluid domain relative to the positions of solid particles and walls as in Fig. 3. At the location of these ghost nodes, the interpolation velocity of fluid particles is determined using kernel interpolation, which serves as the basis for computing the velocity of the solid walls. It should be noted that the smoothing length employed in the kernel interpolation at the ghost nodes ( $h_{GN}$ ) may be defined either equivalently or distinctively from the computational smoothing length ( $h$ ). In this study, the  $h_{GN} = 0.4h$ .

$$\mathbf{u}_{i,solid} = 2U_{wall} - \frac{\sum_{j \in \Omega_f} \mathbf{u}_j W(\mathbf{r}_{GN} - \mathbf{r}_j, h)}{\sum_{j \in \Omega_f} W(\mathbf{r}_{GN} - \mathbf{r}_j, h)} \quad (37)$$

### 3.3.3 Generalized boundary condition (GBC) [50]

The third approach, known as the generalized boundary condition (GBC), directly computes the smoothed velocity field of the fluid phase at the positions of solid particles and applies

it as the velocity of the solid particles. This approach eliminates the need for explicit information regarding boundaries and is particularly advantageous for arbitrary boundaries.

$$\mathbf{u}_{i,solid} = 2U_{wall} - \frac{\sum_{j \in \Omega_f} \mathbf{u}_j W(\mathbf{r}_i - \mathbf{r}_j, h)}{\sum_{j \in \Omega_f} W(\mathbf{r}_i - \mathbf{r}_j, h)}. \quad (38)$$

### 3.3.4 Continuous boundary force method (CBF) [51]

The fourth approach involves a modified version of the continuous boundary force method (CBF), which presents a typical Navier slip boundary condition. In this method, Neumann and Robin boundary conditions at the surface are converted into volumetric forces. To achieve this transformation, an interfacial function ( $\delta_V$ ) is multiplied by the relevant equations, and the interfacial function can be obtained based in kernel gradient [52]. This additional acceleration term is then added to the momentum equation of fluid particles.

$$\mathbf{a}_{CBF} = -\beta \delta_V \mathbf{u}_i \quad (39)$$

$$\delta_V = 2 \left| \sum_{j \in \Omega_f} V_j \nabla_i W_{ij} \right| \quad (40)$$

Here,  $\beta = \mu/b$  represents the friction coefficient, which is a property of the fluid–solid interface, and  $b$  is the slip length. Thus, when  $\beta$  is 0, it implies a free slip condition, while divergence to infinity signifies a no-slip condition. However, for computational efficiency,  $\beta$  can be chosen as a sufficiently large value.

These boundary conditions are integrated into the implicit scheme by modifying the matrix system. Further implementation details are described in Sect. 4.3.

The generalized boundary condition is implemented to apply wall boundary pressure, ensuring a pressure Neumann boundary condition near the wall [50].

$$P_{i,solid} = \frac{\sum_{j \in \Omega_f} P_j W_{ij} + \mathbf{g} \cdot \sum_{j \in \Omega_f} \rho_j \mathbf{r}_{ij} W_{ij}}{\sum_{j \in \Omega_f} W_{ij}} \quad (41)$$

## 4 Implementation of implicit viscosity solvers for linear systems

In this study, we employ a range of iterative methods to evaluate the convergence and performance of the implicit viscosity solvers. Both stationary methods (Jacobi and SOR) and Krylov subspace methods (CG and BiCGSTAB), with

and without Jacobi preconditioning, are tested. All solvers are implemented using a matrix-free approach. This section summarizes the key implementation details.

#### 4.1 Matrix free implementation of stationary iterative method

Firstly, the simplest iterative method, the Jacobi method is considered. As explained earlier, the coefficient matrix at  $n$ -th step  $\mathbf{A}^n$  is known to exhibit SDD characteristics, facilitating convergence through Jacobi iterations. The matrix  $\mathbf{A}^n$  is decomposed into its diagonal component  $\mathbf{D}^n$ , its lower triangular component  $\mathbf{L}^n$ , and its upper triangular component  $\mathbf{U}^n$ . In the Jacobi method, the solution can be obtained iteratively by [28]:

$$\mathbf{x}_{k+1}^n = (\mathbf{D}^n)^{-1}(\mathbf{b} - (\mathbf{L}^n + \mathbf{U}^n)\mathbf{x}_k^n) = (\mathbf{D}^n)^{-1}(\mathbf{b}^n - (\mathbf{A}^n - \mathbf{D}^n)\mathbf{x}_k^n). \quad (42)$$

Here, the  $\mathbf{x}_k^n$  indicates the solution vector at  $k$ -th iteration of the  $n$ -th time-step. Applying this to Eq. (33), it can be expressed as follows:

$$\begin{aligned} u_{i,k}^{n+1} &= \frac{1}{\left[1 - \sum_j K_{ii}^n \Delta t\right]} \left[ u_i^{**} - \sum_{j=1}^N K_{ij}^n u_{j,k}^n \Delta t \right] \\ &= u_i^{**} + \frac{\sum_j K_{ij}^n (u_i^{**} - u_{j,k}^n) \Delta t}{\left[1 - \sum_j K_{ii}^n \Delta t\right]}. \end{aligned} \quad (43)$$

However, the convergence speed of the Jacobi method is known to be very slow. To enhance its convergence speed, the successive over-relaxation (SOR) method is utilized. In the SOR method, the iterative solution of the next iteration step  $\mathbf{x}_{k+1}^n$  can be obtained by weighted averaging of the old step solution  $\mathbf{x}_k^n$  and the temporary solution  $\mathbf{x}_{k+1}^{n,new}$  obtained using the Jacobi method with Eq. (43).

$$\mathbf{x}_{k+1}^n = \omega \mathbf{x}_{k+1}^{n,new} + (1 - \omega) \mathbf{x}_k^{n,old}, (1 < \omega < 2) \quad (44)$$

It is known that the convergent solution can be obtained with fewer iterative steps, leading to faster convergence. The weighting factor  $\omega$  is set to be 4/3 in this study.

#### 4.2 Matrix free implementation of Krylov subspace methods

The coefficient matrix  $\mathbf{A}^n$ , in Eq. (35), is strictly diagonally dominant (SDD), which ensures the convergence of iterative solvers. If the fluid has uniform density and mass, the matrix

also becomes symmetric and positive definite, allowing the use of the CG method. However, the original formulation involving  $\rho_i$  can break symmetry in some cases like multi-phase flow. In such cases, solvers like BiCGSTAB or GMRES for non-symmetric matrices may be used, through they generally involve higher computational cost and implementation complexity. To preserve symmetry, we adopt a modified formulation of the coefficient matrix using the kinematic viscosity ( $\nu = \mu/\rho$ ) [46].

$$K_{ij}^n := \frac{\overline{v}_{ij}^n 2\mathbf{r}_{ij} \cdot \nabla_i W_{ij} V_j}{|\mathbf{r}_{ij}|^2 + \eta^2} \quad (45)$$

This adjustment facilitates much lighter iterative methods for the symmetric matrices, if volume remains relatively constant at a uniform resolution. This formulation has been adopted in this study.

Figure 4 outlines the CG and BiCGSTAB algorithms. Among them, the most computationally demanding operation is the sparse matrix–vector multiplication (SPMV). In general way, this requires pre-computing and storing sparse matrices in compressed formats, which complicates GPU parallelization [38]. In contrast, matrix-free implementation avoids explicit matrix storage. Instead, SPMV is performed directly from particle interactions, as in typical SPH summations. This reduces memory usage, simplifies parallelization, and scales naturally with the number of particles. The computation of  $(\mathbf{A}\mathbf{p})^n$  is expressed as:

$$(\mathbf{A}\mathbf{p})_i^n = \left(1 - \sum_j K_{ii}^n \Delta t\right) p_i^n + \sum_{i \neq j} K_{ij}^n \Delta t p_j^n \quad (46)$$

The final coefficient matrix becomes diagonally dominant due to the restructuring strategy. Thanks to its characteristics, a Jacobi preconditioner is used to accelerate convergence in both CG and BiCGSTAB algorithms within a matrix-free framework. In the preconditioned CG, as illustrated in Fig. 5, the matrix  $\mathbf{M}$  is replaced by the diagonal component matrix  $\mathbf{D}^n$ . This substitution introduces one additional SPMV operation compared to the non-preconditioned CG, and two additional SPMVs in the case of BiCGSTAB. These operations are also implemented in a matrix-free manner, as described below:

$$(\mathbf{M}^{-1} \mathbf{r}_{k+1})_i = r_{k+1,i} / \left(1 - \sum_j K_{ii}^n \Delta t\right) \quad (47)$$

#### 4.3 Matrix construction with boundary decoupling

Wall boundary conditions can be incorporated into implicit time integration in two ways: externally [16] or internally

```

1  function Conjugate Gradient ( $A, b, x_0$ )
2   $r_0 := b - Ax_0$ ;  $p_0 := r_0$ 
3   $k = 0$ 
4  while( $\|r_k\| < \tau$ )
5      Computation  $s_k := Ap_k$ 
6      Reduction ( $r_k, r_k$ ); ( $p_k, s_k$ )
7       $\alpha_k := (r_k, r_k)/(p_k, s_k)$ 
8       $x_{k+1} := x_k + \alpha_k p_k$ 
9       $r_{k+1} := r_k - \alpha_k s_k$ 
10     Reduction ( $r_{k+1}, r_{k+1}$ );
11      $\beta_k := (r_{k+1}, r_{k+1})/(r_k, r_k)$ 
12      $p_{k+1} := r_{k+1} + \beta_k p_k$ 
13      $k++$ 
14 end while
15 end function

```

```

1  function BICGSTAB ( $A, b, x_0$ )
2       $r_0 := b - Ax_0$ ;  $p_0 := r_0$ ;
3       $\hat{r}_0$  is an arbitrary vector which satisfies
4       $(\hat{r}_0, r_0) \neq 0$ , e.g.,  $\hat{r}_0 = r_0$ ;
5       $k = 0$ 
6      while( $\|r_k\| < \tau^2$ )
7          Computation  $s_k := Ap_k$ 
8          Reduction ( $\hat{r}_0, r_k$ ); ( $\hat{r}_0, s_k$ )
9           $\alpha_k := (\hat{r}_0, r_k)/(\hat{r}_0, s_k)$ 
10          $q_k := r_k - \alpha_k s_k$ 
11         Computation  $y_k := Aq_k$ 
12         Reduction ( $q_k, y_k$ ); ( $y_k, y_k$ )
13          $\omega_k := (q_k, y_k)/(y_k, y_k)$ 
14          $x_{k+1} := x_k + \alpha_k p_k + \omega_k q_k$ 
15          $r_{k+1} := q_k - \omega_k y_k$ 
16         Reduction ( $\hat{r}_0, r_{k+1}$ )
17          $\beta_k := (\alpha_k/\omega_k)(r_0, r_{k+1})/(r_0, r_k)$ 
18          $p_{k+1} := r_{k+1} + \beta_k(p_k - \omega_k s_k)$ 
19     end while
20 end function

```

**Fig. 4** Algorithms of CG and BiCGSTAB

within the matrix system [17]. While the latter approach can be more robust and may improve convergence across the entire domain, incorporating arbitrary boundary conditions directly into the matrix is often difficult in SPH due to geometric complexity. Moreover, the symmetry of the matrix can be lost, requiring the use of more computationally intensive iterative solvers.

To preserve matrix symmetry and accommodate arbitrary boundary conditions, we reconstruct the coefficient matrix using only fluid particles. The influence of solid particles is computed separately and incorporated into the right-hand side vector. This decoupled approach simplifies matrix assembly, maintains structural consistency, and allows complex boundary conditions to be handled without directly modifying the matrix system. It also ensures that the wall boundary influences are fully reflected in the solution process, albeit indirectly, via the updated right hand side.

The resulting coefficient matrix  $\mathbf{A}^n = [a_{ij}^n]$  and right-hand side vector are defined as:

$$\mathbf{A}^n = [a_{ij}^n]_{i \in \Omega_f} = \begin{cases} \left[ 1 - \sum_{j \in \Omega_f} K_{ii}^n \Delta t \right] & i = j (\text{diagonal term}) \\ K_{ij}^n \Delta t & i \neq j, j \in \Omega_f \\ 0 & i \neq j, j \notin \Omega_f \end{cases} \quad (48)$$

$$\mathbf{b} = \mathbf{u}_i^{**} - \sum_{j \in \Omega_s} a_{ij}^n \mathbf{u}_j^n. \quad (49)$$

By modifying the coefficient matrix as described above and shown in Fig. 6, our approach preserves the symmetric properties of the matrix even under complex boundary conditions. Furthermore, the matrix retains its strictly diagonally dominant characteristics, which ensures numerical stability and guarantees convergence of iterative solvers. This symmetry, together with the preserved diagonal dominance, enables the use of efficient and robust iterative solvers for symmetric systems such as the conjugate gradient method, resulting in improved computational efficiency. Although the strategy may seem straightforward and may have been implicitly adopted in some earlier works, to the best of our knowledge it has not been systematically described. Moreover, the method

```

1  function Preconditioned CG ( $A, b, x_0$ )
2   $r_0 := b - Ax_0$ ;
3   $z_0 = M^{-1}r_0$ ;  $p_0 := z_0$ ;
4   $k = 0$ 
5  while( $\|r_k\| < \tau$ )
6      Computation  $s_k := Ap_k$ 
7      Reduction ( $r_k, z_k$ ); ( $p_k, s_k$ )
8       $\alpha_k := (r_k, z_k)/(p_k, s_k)$ 
9       $x_{k+1} := x_k + \alpha_k p_k$ 
10      $r_{k+1} := r_k - \alpha_k s_k$ 
11     Computation  $z_{k+1} := M^{-1}r_{k+1}$ 
12     Reduction ( $r_{k+1}, z_{k+1}$ );
13      $\beta_k := (r_{k+1}, z_{k+1})/(r_k, z_k)$ 
14      $p_{k+1} := z_{k+1} + \beta_k p_k$ 
15 end while
16 end function

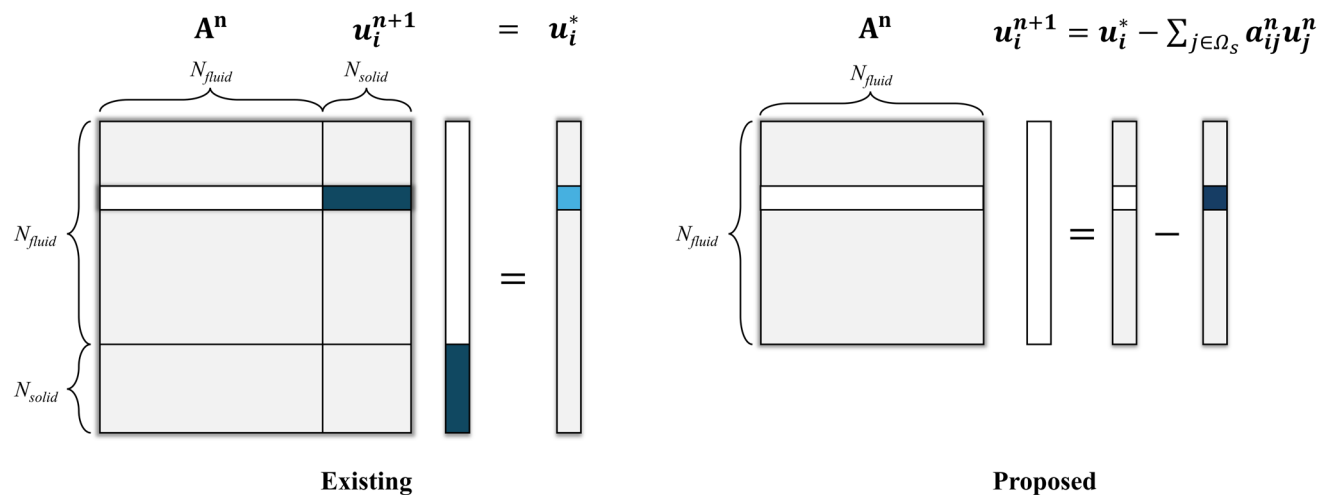
```

```

1  function Preconditioned BICGSTAB ( $A, b, x_0$ )
2       $r_0 := b - Ax_0$ ;  $p_0 := r_0$ ;
3       $\widehat{r}_0$  is an arbitrary vector which satisfies
4       $(\widehat{r}_0, r_0) \neq 0$ , e.g.,  $\widehat{r}_0 = r_0$ ;
5       $k = 0$ 
6      while( $\|r_k\| < \tau^2$ )
7          Computation  $\widehat{p}_k := M^{-1}p_k$ 
8          Computation  $s_k := A\widehat{p}_k$ 
9          Reduction ( $r_0, r_k$ ); ( $r_0, s_k$ )
10          $\alpha_k := (r_0, r_k)/(r_0, s_k)$ 
11          $q_k := r_k - \alpha_k s_k$ 
12         Computation  $\widehat{q}_k := M^{-1}q_k$ 
13         Computation  $y_k := A\widehat{q}_k$ 
14         Reduction ( $q_k, y_k$ ); ( $y_k, y_k$ )
15          $\omega_k := (q_k, y_k)/(y_k, y_k)$ 
16          $x_{k+1} := x_k + \alpha_k \widehat{p}_k + \omega_k \widehat{q}_k$ 
17          $r_{k+1} := q_k - \omega_k y_k$ 
18         Reduction ( $r_0, r_{k+1}$ )
19          $\beta_k := (\alpha_k/\omega_k)(r_0, r_{k+1})/(r_0, r_k)$ 
20          $p_{k+1} := r_{k+1} + \beta_k(p_k - \omega_k s_k)$ 
21 end function

```

**Fig. 5** Algorithm of preconditioned-CG (PCG) and preconditioned-BiCGSTAB (PBiCGSTAB)



**Fig. 6** Conceptual representation of coefficient matrix restructuring

is not restricted to the present application; it can be readily extended to problems that require solving matrices with arbitrary boundary treatments across diverse physical settings.

#### 4.4 GPU parallelization

In this study, the modified linked list technique was employed to perform the neighbor search for SPH particles [53, 54]. This method is widely recognized for its efficiency and low memory usage, making it a popular choice for reducing the computational cost of neighbor search in particle methods. The computational domain is divided into fixed cells with a width of the smoothing function,  $\kappa h$  as in Fig. 7. All potential neighbors of a particle are located within its cell and the immediately adjacent cells. Each cell is assigned a unique index to efficiently organize and manage particle data.

At each simulation time step, the cell index for each particle is calculated, and particles are sorted based on these indices. A particle index (PID) list is generated to maintain this sorted order, and particle data is rearranged accordingly. This ensures that particles within the same cell are stored in contiguous memory locations, improving memory access efficiency, as illustrated in Fig. 8.

The neighbor search is carried out using thread-based parallelization, where each target particle is assigned to a single thread. In each thread, the neighbor search involves iterating through particles in the same cell and neighboring cells. For each candidate neighbor, the distance is calculated to determine whether it lies within the smoothing range. While this approach may not achieve the highest computational efficiency, it is designed to maximize thread utilization, making it particularly suitable for simulations involving large numbers of particles. In this study, we performed our study based on the open-source code SOPHIA [55]. While this method has been extended to support multi-GPU [55] and MPI [54] for large-scale simulations, we focused solely on a single GPU implementation. The system utilized for this study was equipped with an Intel(R) Xeon(R) CPU E5-2650 v4 @ 2.20GHz, a Tesla P100 GPU with 16GB of memory, and Ubuntu 18.04.6 LTS as the operating system.

## 5 Numerical benchmarks

To assess the accuracy and robustness of the implicit viscosity solver used in this study, we conducted verification tests involving two steady-state cases and one transient case.

#### Case 1. Two-Dimensional Newtonian Poiseuille Flow

- A. Single-phase flow
- B. Multiphase flow (with viscosity contrast)

#### Case 2. Two-Dimensional Non-Newtonian Poiseuille Flow

- A. Shear-thinning fluid
- B. Shear-thickening fluid
- C. Bingham fluid
- D. Herschel–Bulkley fluid

#### Case 3. Two/Three-Dimensional Isothermal Viscous Spreading

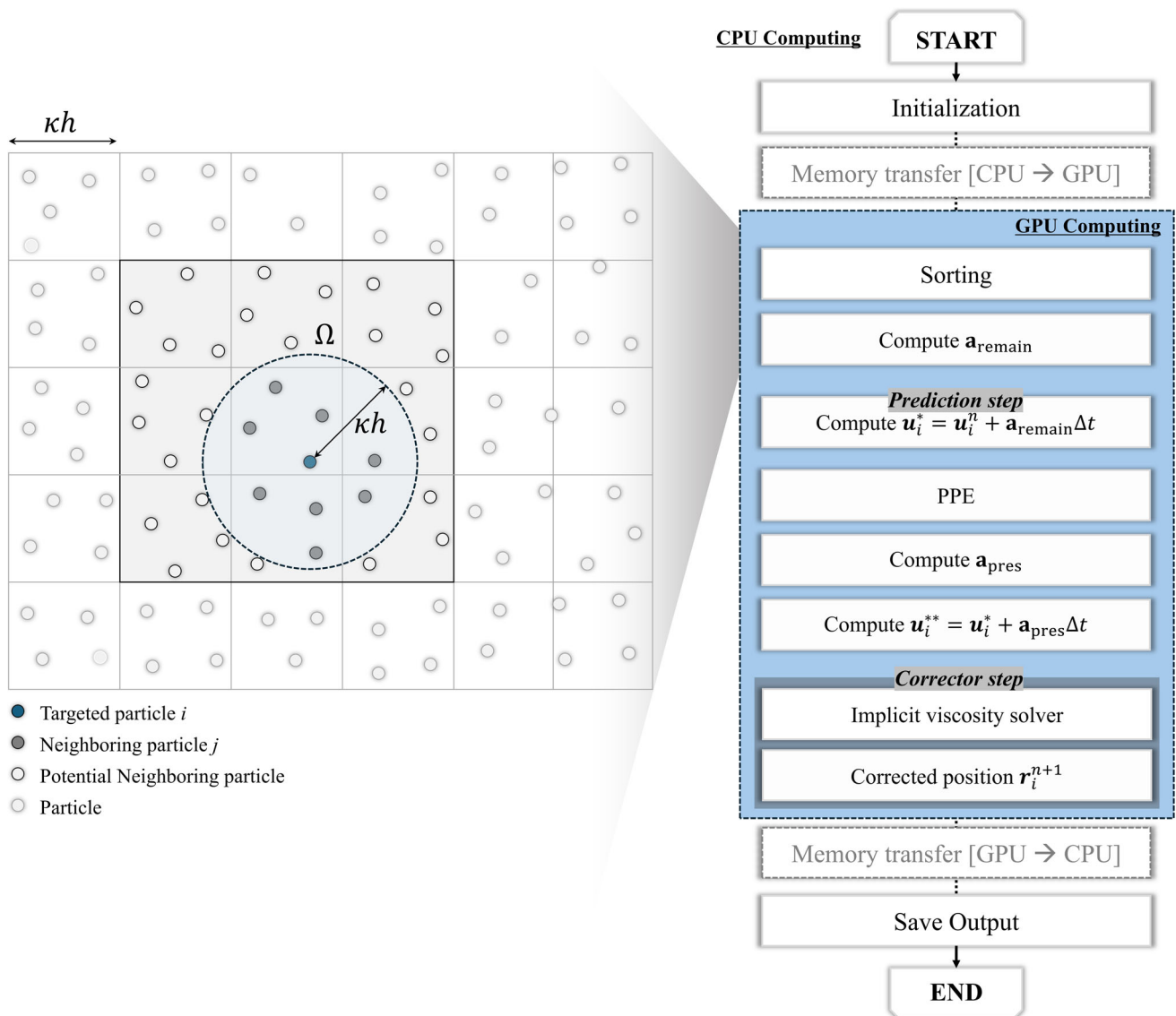
For the steady-state analyses, we investigated pipe flows under both uniform and variable viscosity conditions. This included single-phase Newtonian flows and multi-phase cases with a viscosity contrast of 100. These tests allowed for a comprehensive evaluation of the solver's accuracy, convergence behavior, and computational performance under various boundary condition treatments and iterative solver strategies. In the non-Newtonian flow cases, four representative rheological models were used to examine solver behavior across a range of shear-dependent viscosity conditions. Finally, the solver's transient performance was assessed through isothermal viscous spreading simulations in both two and three dimensions, providing insight into temporal stability and accuracy.

## 5.1 Two-dimensional single/multi-phase Newtonian Poiseuille flow

### 5.1.1 Numerical condition

For the first scenario, we analyze pipe flow with uniform viscosity. The initial particle configuration, as illustrated in Fig. 9a, features a 2D pipe flow with a width  $D$  and includes two-sided walls where no-slip boundary conditions are enforced. To achieve fully developed flow, periodic boundary conditions are applied at both the inlet and outlet [56]. Initially, we consider a single-phase case with a uniform viscosity of  $1\text{ Pa}\cdot\text{s}$  and a fluid density set at  $1\text{ kg/m}^3$ . A body force  $f_y = 2\text{ m/s}^2$  induces the fluid motion. We assess three different spatial resolutions:  $\Delta x = D/20$  (coarse),  $D/40$  (medium), and  $D/80$  (fine), where  $D$  is 1 m in this study. The time step is set to 0.01 s for all simulations.

In the second scenario, we explore the dynamics of two fluids with differing constant viscosities, as shown in Fig. 9b. Along with the vertical axis, fluid 1 occupies the left side, and fluid 2 the right, with viscosities of  $1\text{ Pa}\cdot\text{s}$  and  $100\text{ Pa}\cdot\text{s}$ , respectively. The time-step for both fluids is set at 0.01 s and total simulation time is 2 s. Firstly, for both scenarios, the no-slip boundary conditions are an enforced method. We compared numerical results from both scenarios with analytic solution



**Fig. 7** Overview of the overall computing order

as follows [57]:

$$u_{y,1} = f_y \frac{(D/2)^2}{2\mu_1} \left[ \left( \frac{2\mu_1}{\mu_1 + \mu_2} \right) + \left( \frac{\mu_1 - \mu_2}{\mu_1 + \mu_2} \right) \left( \frac{x - D/2}{D} \right) - \left( \frac{x - D/2}{D} \right)^2 \right], \quad (50)$$

$$u_{y,2} = f_y \frac{(D/2)^2}{2\mu_2} \left[ \left( \frac{2\mu_2}{\mu_1 + \mu_2} \right) + \left( \frac{\mu_1 - \mu_2}{\mu_1 + \mu_2} \right) \left( \frac{x - D/2}{D} \right) - \left( \frac{x - D/2}{D} \right)^2 \right]. \quad (51)$$

Here,  $u_{y,1}$  and  $u_{y,2}$  denotes the velocity profile of fluid 1 and fluid 2,  $\mu_1$  and  $\mu_2$  indicates the viscosity of fluid 1 and fluid 2. For the first scenario, these are identical.

### 5.1.2 Comparison of iterative solvers

Initially, we examined the convergence rate at the first step for several iterative solvers (Jacobi, SOR, CG, PCG, BiCGSTAB, PBiCGSTAB) across two cases, with a fine resolution. In this analysis, the boundary condition was consistently applied using the FGP method. As shown in Fig. 10, the convergence rates of Krylov methods were significantly faster than stationary methods. This difference was particularly pronounced in the multi-phase scenario compared to the single-phase with uniform viscosity. Additionally, while the use of a preconditioner in the single-phase case did not significantly improve performance, in the multi-phase case, applying a preconditioner reduced the number of iterations to convergence by approximately 3.5 and 4 times for CG

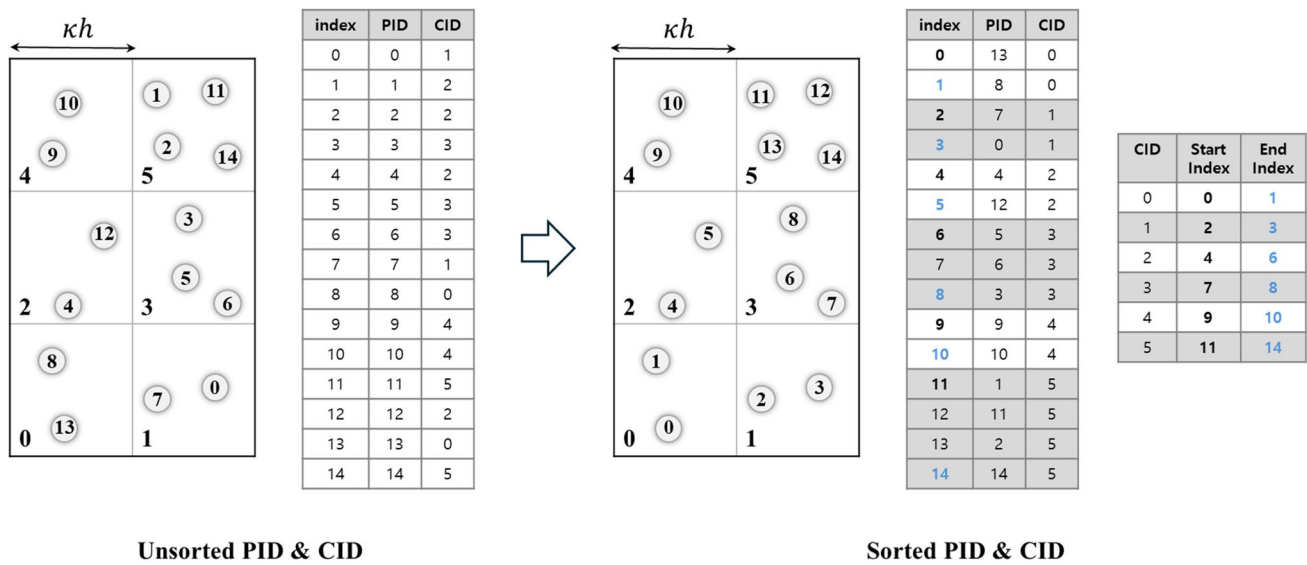


Fig. 8 Neighboring particle search algorithm

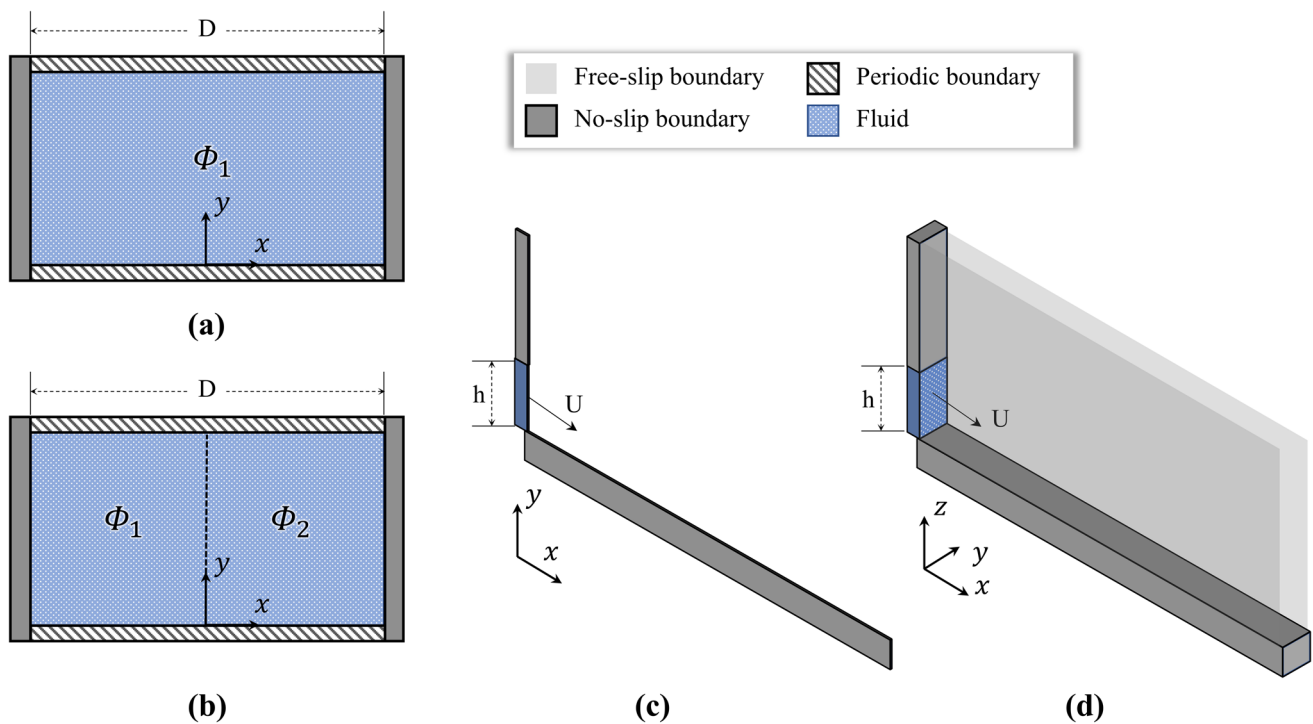


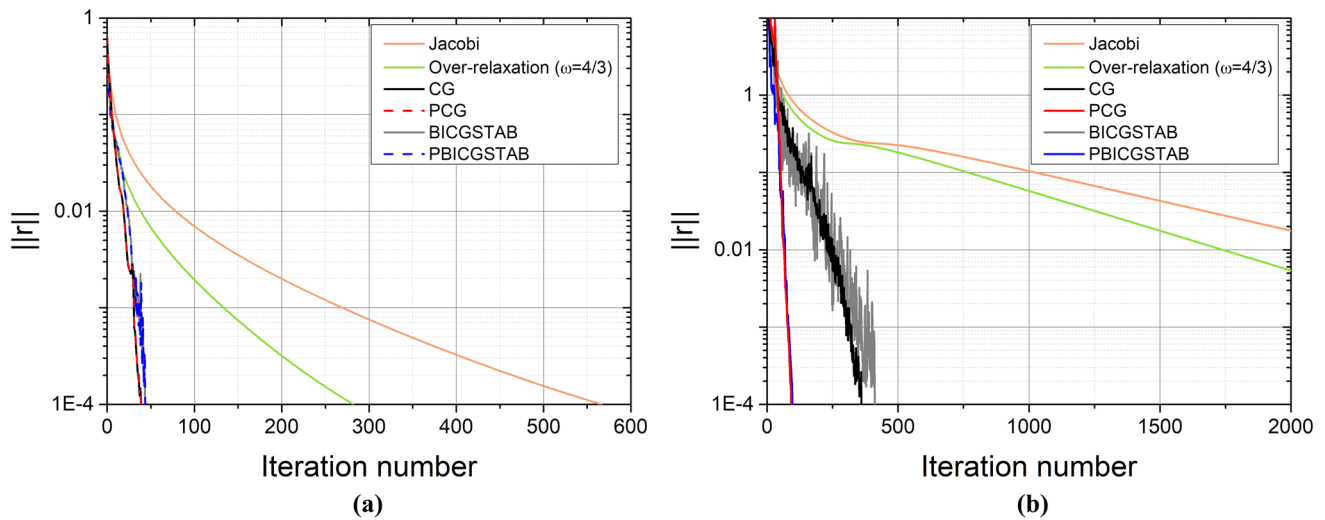
Fig. 9 Initial geometrical configurations and boundary conditions of numerical benchmarks

and BiCGSTAB, respectively. Notably, the use of a Jacobi preconditioner with BiCGSTAB substantially improved the stability of residual convergence.

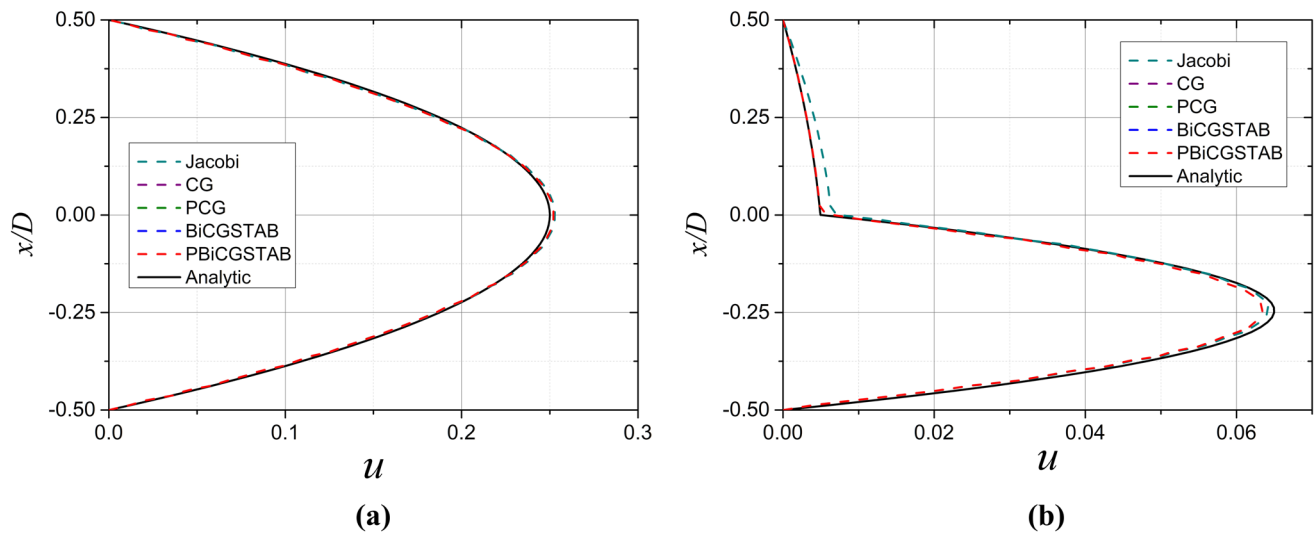
The velocity profiles at steady state were compared with the analytical solution. As shown in Fig. 11, all solvers exhibited results that were very close to the analytical solution for both scenarios.

Additionally, the total computing times of explicit and implicit solvers for both scenarios are compared in Fig. 12. As

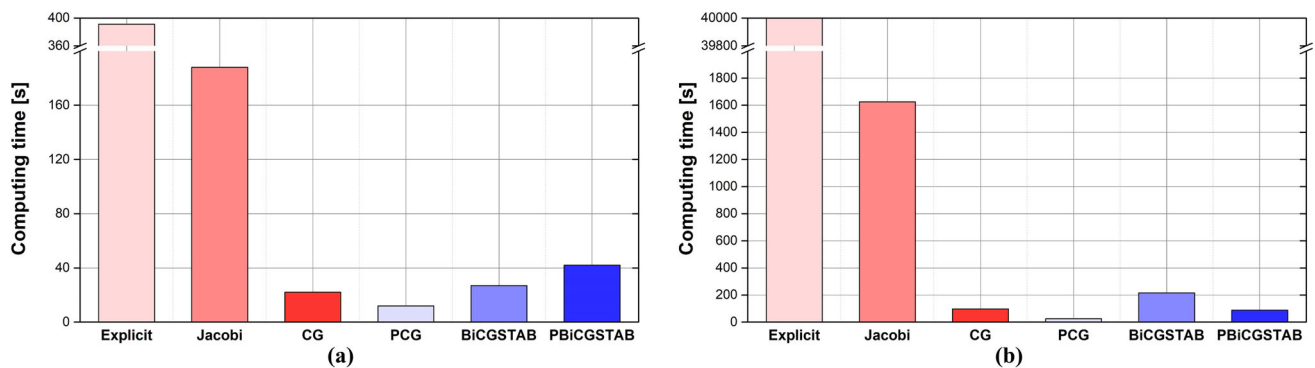
expected, implicit solvers demonstrated a significant reduction in computing time compared to the explicit approach. In the single-phase scenario, consistent with the minimal variation in iteration numbers among the four Krylov methods as illustrated in Fig. 12, algorithms with greater complexity required longer computing times. For instance, although PBiCGSTAB and PCG had similar iteration numbers, the analysis time for PBiCGSTAB was approximately twice that of PCG.



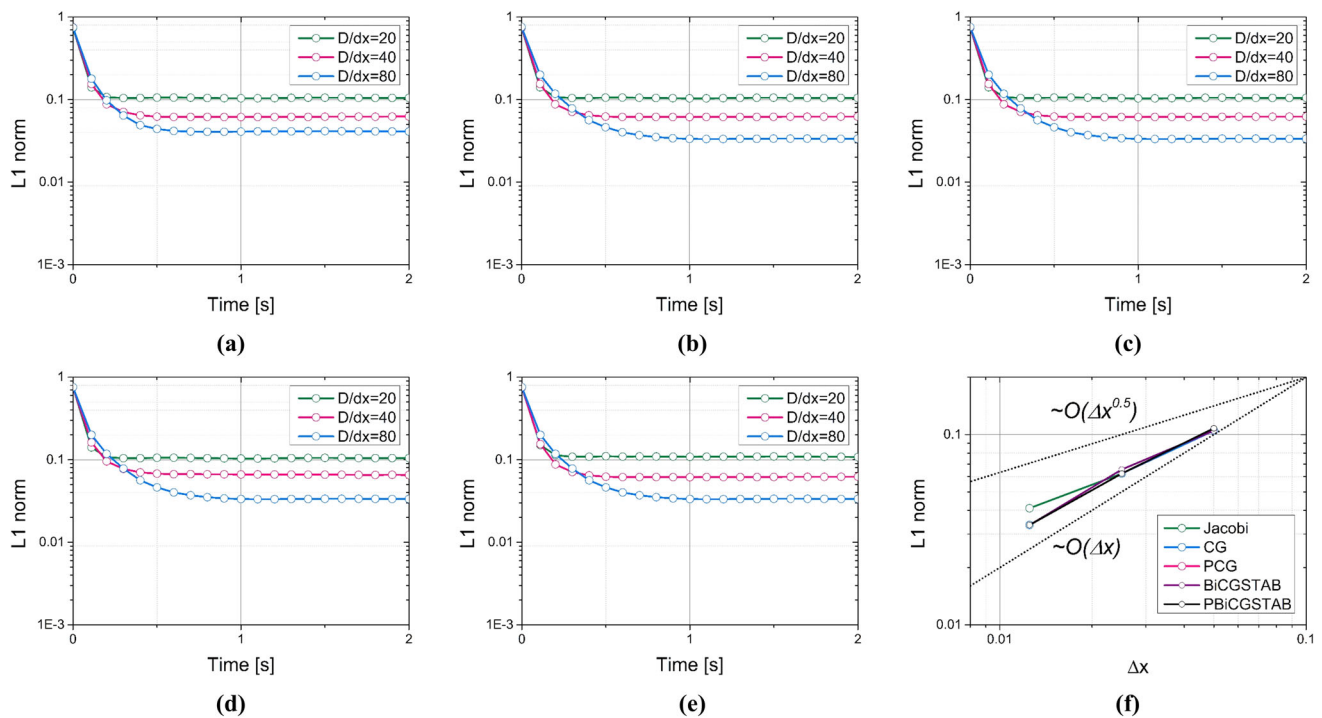
**Fig. 10** Comparative Results of residual convergence in various iterative methods for two-dimensional Newtonian Poiseuille Flow: **a** single-phase and **b** multi-phase scenarios



**Fig. 11** Comparative results of velocity contour in various iterative methods for two-dimensional Newtonian Poiseuille flow: **a** single-phase and **b** multi-phase scenarios



**Fig. 12** Comparative results of computing time in various iterative methods for two-dimensional Newtonian Poiseuille flow at fine resolution: **a** single-phase and **b** multi-phase scenarios



**Fig. 13** Convergence comparison of various iterative methods for two-dimensional multi-phase Newtonian Poiseuille flow: **a** Jacobi **b** CG **c** PCG **d** BiCGSTAB **e** PBiCGSTAB

However, in the multi-phase scenario, the inclusion of a fluid with a viscosity 100 times greater naturally led to a smaller time step requirement for explicit solvers due to time step constraints, showing a substantially greater difference compared to implicit solvers. As a result, PCG achieved up to  $2500 \times$  speed-up over the explicit method. Preconditioning effectively reduced both iteration counts and runtime: iteration counts dropped by factors of  $\sim 3$  (CG) and  $\sim 4$  (BiCGSTAB), with corresponding runtime reductions of  $\sim 4 \times$  and  $\sim 2 \times$ , respectively. These results, while expected, underscore that reformulating the system to enforce matrix symmetry and preserve strict diagonal dominance enables the use of simpler and more efficient solvers such as CG, leading to significant improvements in overall computational performance.

We have now examined the convergence of the L1 norm over time for Jacobi, CG, PCG, BiCGSTAB, and PBiCGSTAB, excluding SOR, across three different resolutions. The convergence rates over time for the Krylov methods were almost identical, while Jacobi showed slightly reduced convergence at finer resolutions as shown in Fig. 13. Overall, we confirmed an order of accuracy between 0.5 and 1 and showed stable convergent behavior.

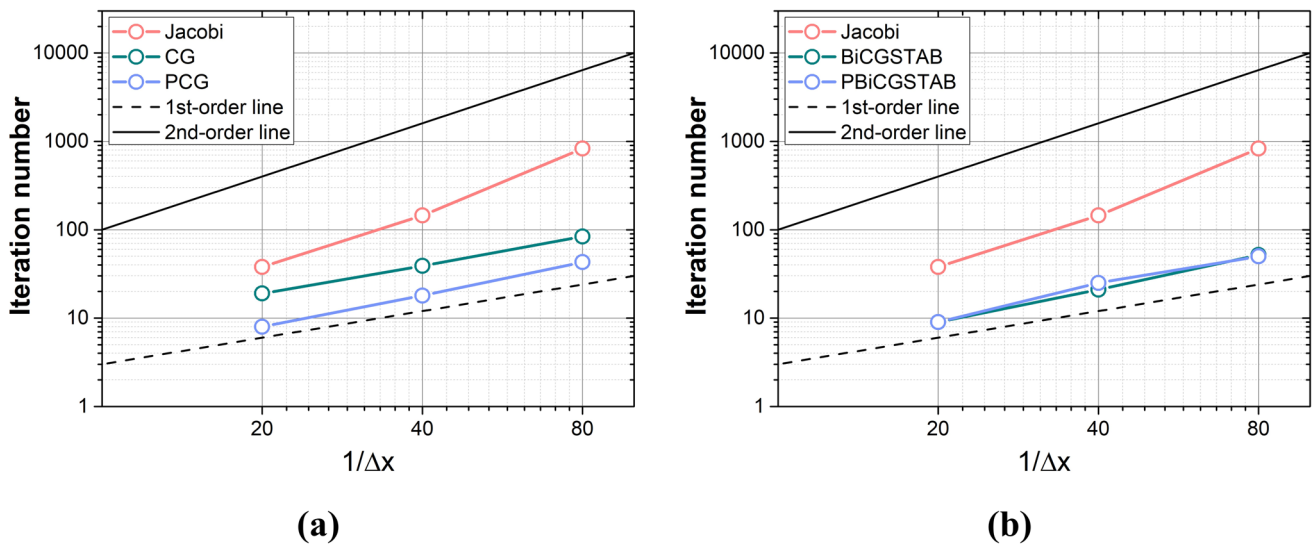
Figures 14 and 15 illustrate the iteration counts required for convergence at final computing step across various iterative methods. Unlike the single-phase case, where the effect of the Jacobi preconditioner was minimal, its impact

was more pronounced in the multi-phase case with a high-viscosity ratio. Here, the preconditioner significantly reduced the absolute iteration count and lowered the rate of increase in iteration count as spatial discretization was refined. Both CG and BiCGSTAB initially showed an increase in iteration count with particle spacing at an order of 1.3, which was reduced to a first-order rate upon applying the Jacobi preconditioner.

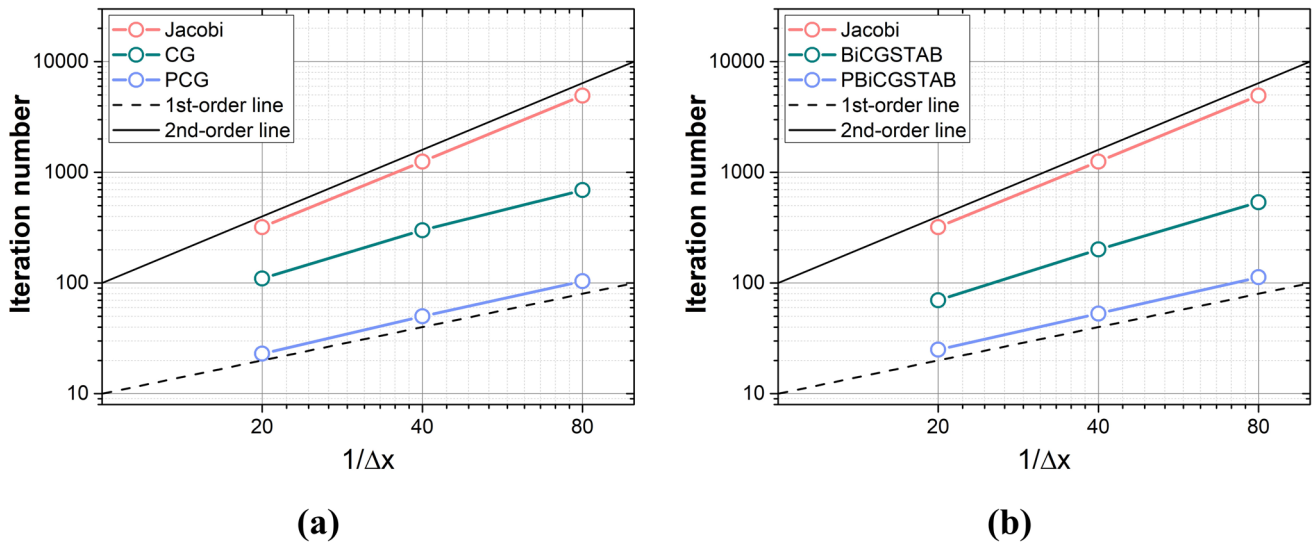
### 5.1.3 Effect of boundary condition

With the PCG solver, multi-phase scenarios are revisited employing various boundary conditions as detailed in Sect. 3.3. Four distinct boundary condition treatments were applied, and the corresponding results are presented in Fig. 16. All simulations achieved convergence over time, with the CBF method yielding the most accurate results. However, the high-resolution case with CBF method required relatively larger computing time to get converged results.

In Fig. 17, overall accuracy with respect to particle resolution and iteration count across resolutions are compared for four boundary condition treatments. As depicted, the FWP, FGP, and GBC methods demonstrated an order of accuracy between 0.5 and 1. Among these, the FWP method exhibited slightly lower errors than FGP and GBC, which produced nearly identical results. In contrast, the CBF method achieved close to second-order accuracy, significantly outperforming



**Fig. 14** Comparison of iterative methods by iteration count required for convergence at increasing spatial resolutions for single phase Newtonian Poiseuille flow: **a** CG with and without Jacobi preconditioner **b** PBiCGSTAB with and without Jacobi preconditioner



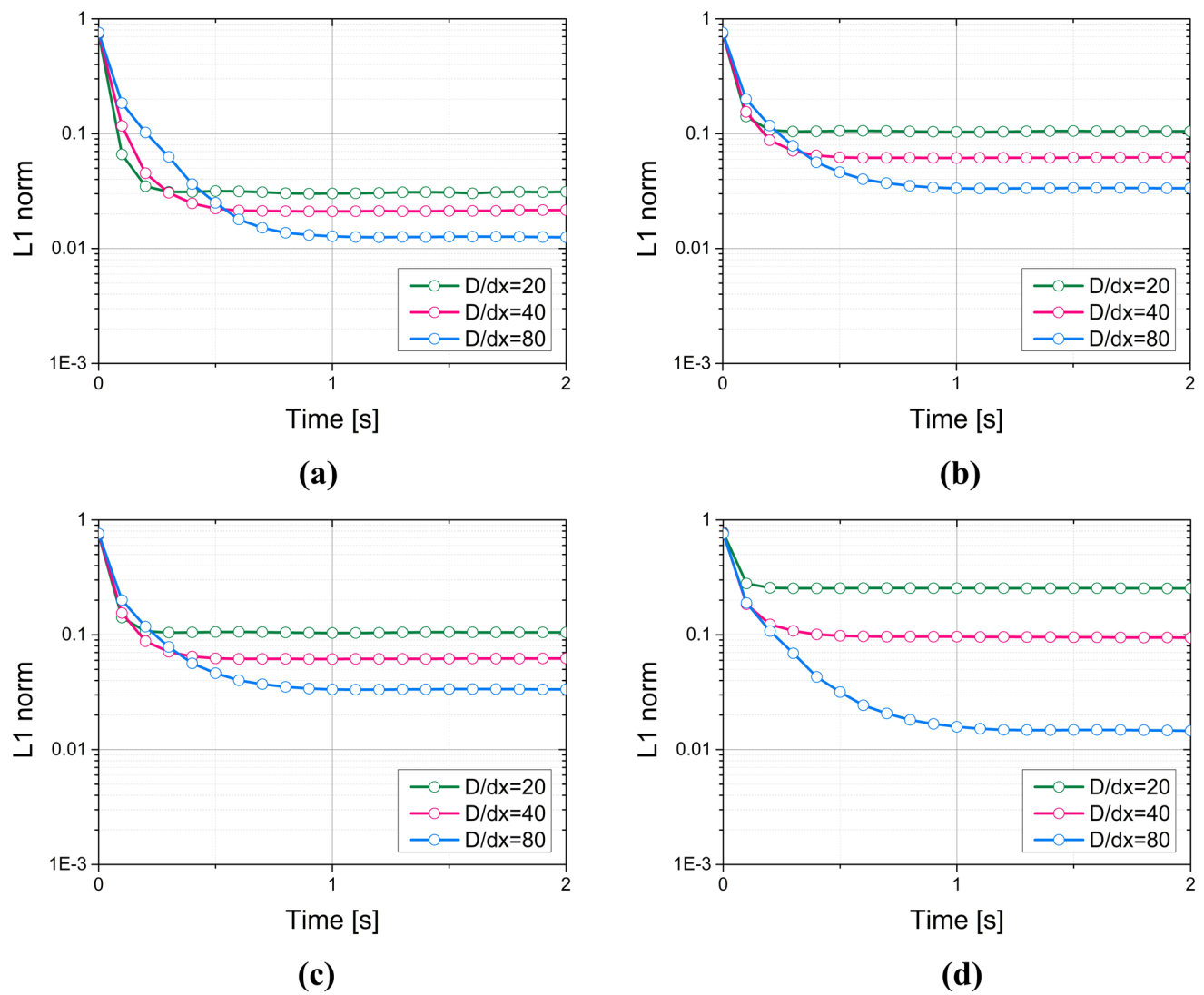
**Fig. 15** Comparison of iterative methods by iteration count required for convergence at increasing spatial resolutions for multi phase Newtonian Poiseuille flow: **a** CG with and without Jacobi preconditioner **b** PBiCGSTAB with and without Jacobi Preconditioner

the others in terms of precision and error reduction. Despite these differences in accuracy, the iteration counts among all four methods were similar, and consequently, there was no noticeable difference in computation time when simulating the same physical duration. While the CBF method was found to be the most accurate, it required significantly more simulation time to reach a converged result. Accordingly, the FGP and GBC method was chosen for the remaining simulations.

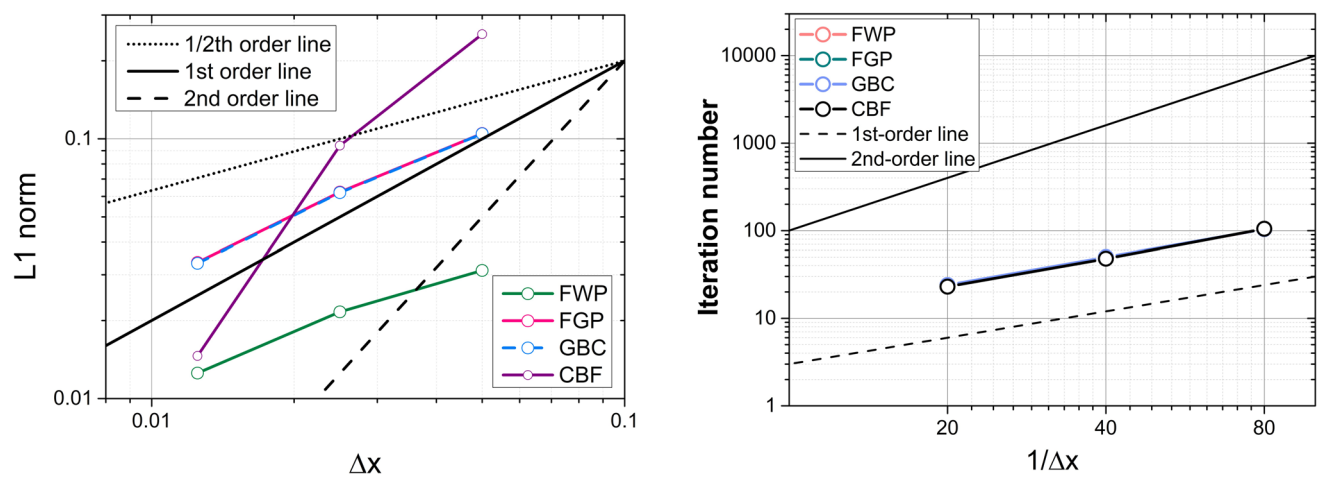
## 5.2 Two-dimensional non-Newtonian Poiseuille flow

### 5.2.1 Numerical modeling on non-Newtonian flows

To assess the capability and performance of our implicit viscosity solver, we consider a range of non-Newtonian rheological models, beginning with the generalized



**Fig. 16** Convergence comparison of various boundary conditions for Newtonian multi-phase fluids with PCG: **a** FWP **b** FGP **c** GBC **d** CBF



**Fig. 17** Performance comparison of various boundary conditions for Newtonian multi-phase fluids with PCG: **a** accuracy **b** iteration count

Herschel-Bulkley model. This model encompasses both yield stress and shear-dependent viscosity, providing a flexible framework that includes power-law and Bingham behaviors as specific cases. Like the previous benchmark case involving Newtonian fluids, a 2D pipe flow with two-sided walls is considered as illustrated in Fig. 9a, with three different spatial resolutions. To deal with non-Newtonian fluid flows same governing equations are used as in the previous benchmark but, as noted in Sect. 2.1 the viscosity has to be replaced with the apparent viscosity,

The Herschel–Bulkley (HB) model is defined as follows:

$$\mu_{\text{eff}} = \begin{cases} \frac{\tau_y}{|\dot{\gamma}|} + \mu_0 |\dot{\gamma}|^{N-1} & \text{if } \tau := \mu |\dot{\gamma}| \geq \tau_y \\ \mu_0 & \text{if } \tau := \mu |\dot{\gamma}| < \tau_y \end{cases} \quad (52)$$

where  $\tau$  is the shear stress,  $\tau_y$  is the yield stress,  $\mu_0$  is the consistency index,  $\mathbf{D}$  is the shear rate, and  $N$  is flow behavior index. Special cases include:

- Power-law model: Setting  $\tau_y = 0$  yields a shear rate-dependent viscosity where:

$$\mu_{\text{eff}} = \mu_0 |\dot{\gamma}|^{N-1} \quad (53)$$

Here, the flow behavior index  $m$  defines the fluid behavior:

- $N = 1$ : Newtonian fluid behavior.
- $N < 1$ : Shear-thinning behavior (pseudoplastic).
- $N > 1$ : Shear-thickening behavior (dilatant).
- Bingham model: Setting  $m = 1$  and introducing a non-zero yield stress  $\tau_y$  captures yield-stress behavior.

The Bingham and Herschel-Bulkley fluids exhibit discontinuous shear rates, which can introduce numerical instability in simulations. To address this, several models have been proposed within the SPH framework [39, 47, 58, 59]. Among these, Hosseini's bilinear model has demonstrated favorable stability and accuracy in recent studies [45, 59], making it a suitable choice for the current study.

$$\mu_{\text{eff}} = \begin{cases} \frac{\tau_y}{|\dot{\gamma}|} + \mu_0 |\dot{\gamma}|^{N-1} & \text{if } \tau := \mu |\dot{\gamma}| \geq \tau_y / \alpha \\ \alpha \mu_0 & \text{if } \tau := \mu |\dot{\gamma}| < \tau_y / \alpha \end{cases} \quad (54)$$

where the penalty coefficient  $\alpha$  is the large number which has order of  $10^2$  [47, 59].

The time step for all simulations is set to  $10^{-4}$  s. In the non-Newtonian case, the analytic solution can be derived as

follows [59]:

$$u_y = \left( \frac{f_y}{\mu} \right)^{\frac{1}{N}} \frac{N}{N+1} \left[ \left( \frac{D}{2} - y_0 \right)^{\frac{1}{N}+1} - (|x| - y_0)^{\frac{1}{N}+1} \right]. \quad (55)$$

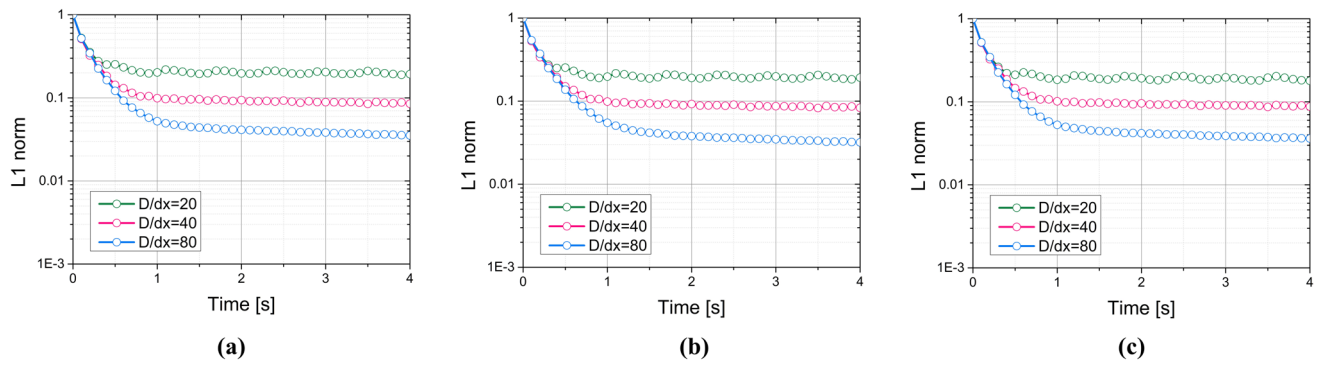
## 5.2.2 Result with different iterative solvers

To evaluate the convergence and accuracy of an implicit viscosity solver on non-Newtonian flows, we initially tested three iterative methods on simple shear-thinning flow with  $N = 0.25$ . The FGP boundary treatment is adopted for this study.

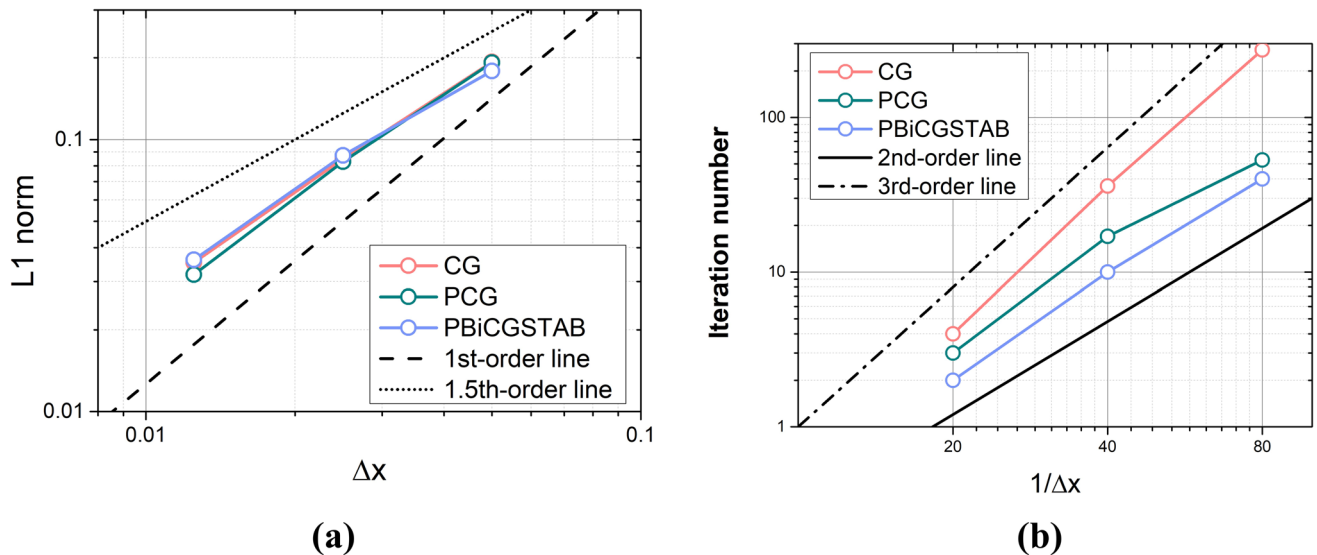
As shown in Fig. 18, the CG, PCG, and PBiCGSTAB methods demonstrated stable convergence over time, with L1norm of each method indicating consistent convergence behavior. In Fig. 19a, the final convergence accuracy of each iterative method is compared, with accuracy values ranging between 1.0 and 1.5. Additionally, Fig. 19b presents the iteration count required for convergence at the final computational step. The CG method shows that the required iteration count increases with third order dependency as the  $\Delta x$  decreases, while the preconditioned methods achieve similar accuracy but with a reduced iteration count increase, following a second-order dependency as the  $\Delta x$  decreases.

## 5.2.3 Numerical results for various non-Newtonian flow rheologies

As the next step, we examined non-Newtonian flows with different rheologies, including shear-thinning, shear-thickening, Bingham, and HB flows, to validate the performance of our implicit viscosity solver across a range of non-Newtonian behaviors. Figure 20 presents the time evolution of the L1-norm for these cases. As shown, both shear-thinning and shear-thickening flows exhibited stable convergence. Notably, the shear-thickening flow was extremely stable across all resolutions, from coarse to fine. In contrast, the shear-thinning flow showed minor oscillations at coarse resolution. In the Bingham flow case, while convergence improved with higher resolutions, slight oscillations were observed at coarse and medium resolutions. These oscillations became more pronounced in the HB case, where shear-thinning behavior was combined with Bingham flow characteristics, and convergence was slightly less stable at fine resolution. In the HB case, where shear-thickening behavior was combined with Bingham flow characteristics, the flow demonstrated high stability, like the shear-thickening case. These instabilities are mainly attributed to the discontinuous yield-stress transition in Bingham and shear-thinning HB models, which induces stiffness at low shear rates,



**Fig. 18** Convergence comparison of various iterative methods for shear-thinning fluids: **a** CG **b** PCG **c** PBiCGSTAB



**Fig. 19** Performance of iterative methods for shear-thickening flow at different spatial resolutions: **a** accuracy **b** iteration count

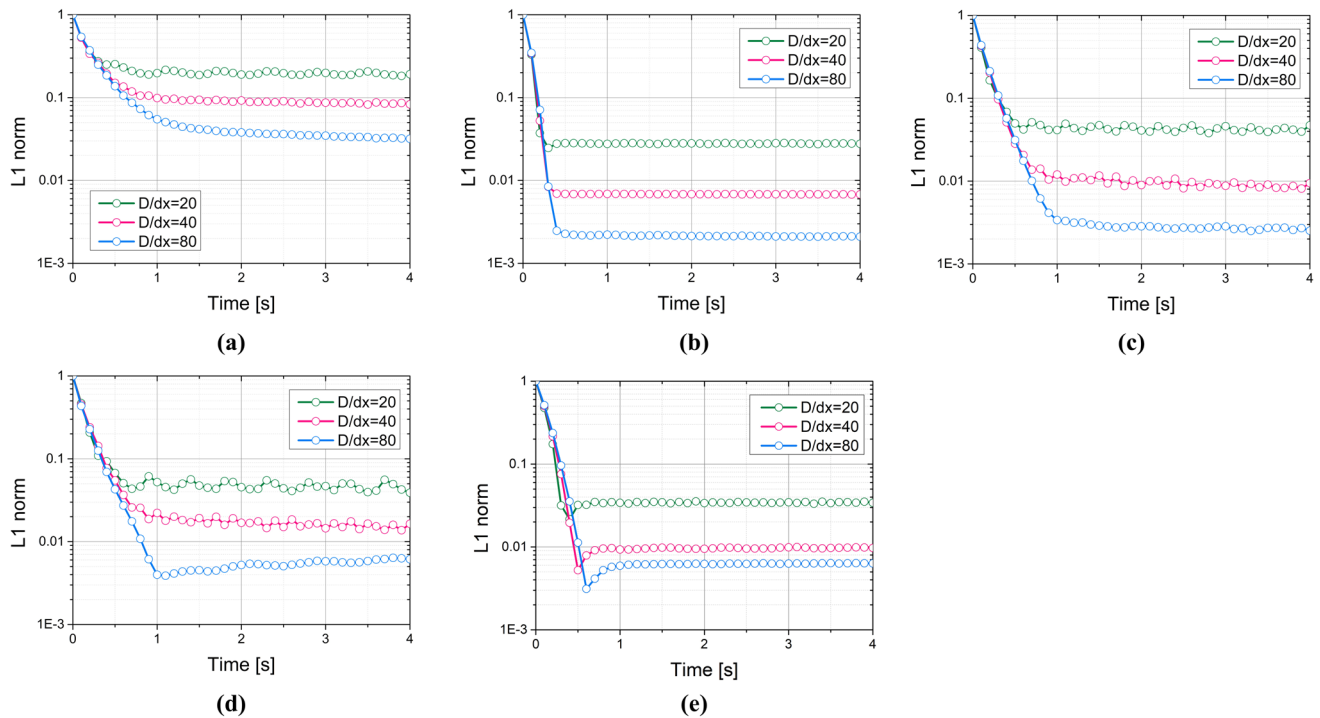
together with resolution sensitivity, where coarse discretization amplifies local shear-rate fluctuations. These instabilities are mainly attributed to the discontinuous yield-stress transition in Bingham and shear-thinning HB models, which induces stiffness at low shear rates, together with resolution sensitivity, where coarse discretization amplifies local shear-rate fluctuations. In shear-thinning HB cases, the convergence at fine resolution may appear to degrade over long simulations, and this behavior is closely related to the choice of the penalty coefficient in the bilinear regularization model. As described in Sect. 5.2.1, the coefficient was set to 100 in this study, following previous literature. However, the selection of these naturally affects the results. A smaller  $\alpha$  can alleviate short-term instabilities at low resolution, but it also distorts the stress–strain curve, leading to reduced accuracy and long-term convergence. Conversely, a larger  $\alpha$  does not completely remove low-resolution instabilities but generally improves overall accuracy and robustness. These observations indicate that the selection of the penalty coefficient  $\alpha$

of the bilinear model should be made in consideration of the specific problem and resolution (Fig. 21).

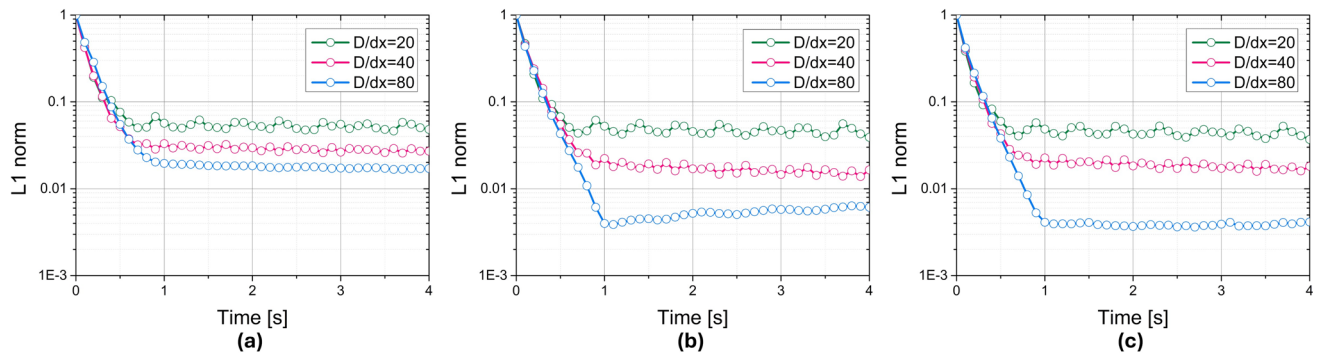
In Fig. 22a, the final convergence accuracy of each iterative method is compared, with accuracy values ranging between 1.0 and 2.0. In Fig. 22b, the iteration count required for convergence at the final computational step is shown for the shear-thinning and HB cases with shear-thinning behavior. The results indicate that the iteration count increases with a second-order dependency as the  $\Delta x$  decreases.

### 5.3 Two/three-dimensional isothermal viscous spreading

In regimes characterized by low Reynolds numbers, where viscous and gravitational forces prevail, the advancing front of a viscous fluid flow in the gravity-viscous domain can be described by an analytical solution. Consider a two-dimensional space wherein a fluid, possessing a time-dependent viscosity  $\mu(= \mu_0 t^\beta)$ , is infused at a volume flux rate of  $\dot{v}(t)(= \alpha q t^{\alpha-1})$  per unit channel width. Here,  $\alpha$  and  $\beta$



**Fig. 20** Convergence comparison for shear-thickening fluids using PCG: **a** shear-thinning **b** shear-thickening **c** Bingham **d** HB with shear-thinning behavior **e** HB with shear-thickening behavior



**Fig. 21** Effect of penalty coefficient  $\alpha$  on HB with shear-thinning behavior at different spatial resolutions: **a**  $\alpha = 10$ , **b**  $\alpha = 100$ , and **c**  $\alpha = 1000$

are constant and  $t$  denotes time. Under these conditions, the position of the spreading front is governed by the analytical relation [60]:

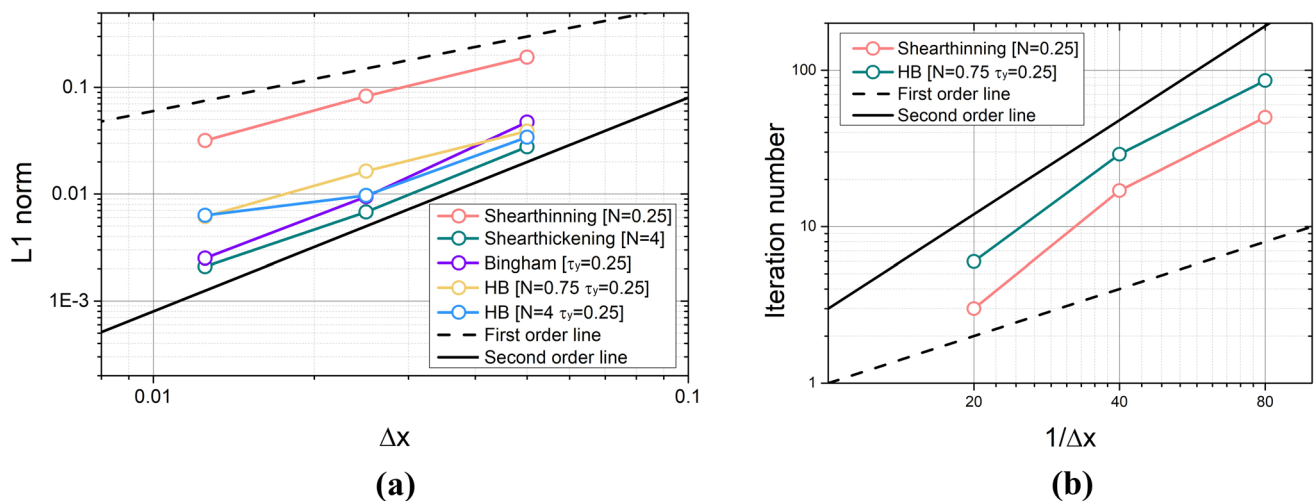
$$x_f = \eta_f \left( \frac{gq^3}{3\mu_0} \right)^{0.2} t^{0.2(3\alpha-\beta+1)} \quad (56)$$

In instances where the fluid is injected with a constant viscosity and at a constant volume flux rate, the values of  $\alpha$  and  $\beta$  default to 1 and zero, respectively. Consequently, the coefficient  $\eta_f$  assumes the value of 0.95963.

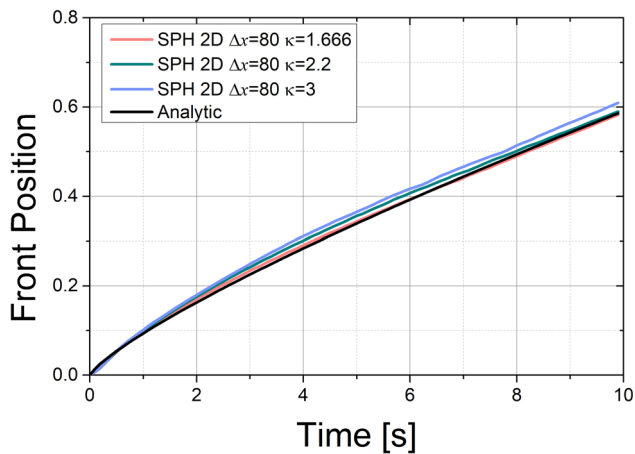
The same two-dimensional spreading test was conducted in both 2D and 3D domains, as illustrated in Fig. 9c and d. In the 3D domain, the lateral boundaries were treated as slip

walls. To model slip walls, these wall particles were excluded from the contribution of the viscous term. To model remaining no-slip walls, the GBC boundary treatment is used to compute wall velocity. The domain height was set at 0.04 m, with a mass flux of 3 kg/s. The fluid, characterized by a density of 1000 kg/m<sup>3</sup> and a viscosity of 10 Pa · s, exhibits a viscosity 10,000 times that of water. Simulations were executed in 2D with grid resolutions of  $h/dx = 20, 40$ , and in 3D with  $h/dx = 10, 20, 40$ .

The position of the spreading front was identified as the highest point of the fluid in contact with the bottom surface. In the 2D simulation, the impact of the smoothing length was evaluated to assess its role in accurately capturing the free surface. Results showed that a smoothing length factor



**Fig. 22** Performance of PCG on various non-Newtonian flows at different spatial resolutions: **a** accuracy **b** iteration count



**Fig. 23** 2D numerical results of isothermal spreading using PCG at  $\Delta x = D/80$  with different smoothing length

of  $\kappa = 1.666$  provided the closest match to the analytical solution in Fig. 23.

Consequently,  $\kappa$  was set to 1.666 for both 2D and 3D simulations in this study. As illustrated in Fig. 24, the numerical results closely align with the analytical solution for both 2D and 3D cases. This agreement demonstrates that the EISPH, enhanced by the implemented implicit viscosity solver, effectively captures both steady-state and dynamic behaviors of viscous fluids (Fig. 25).

## 6 Conclusion

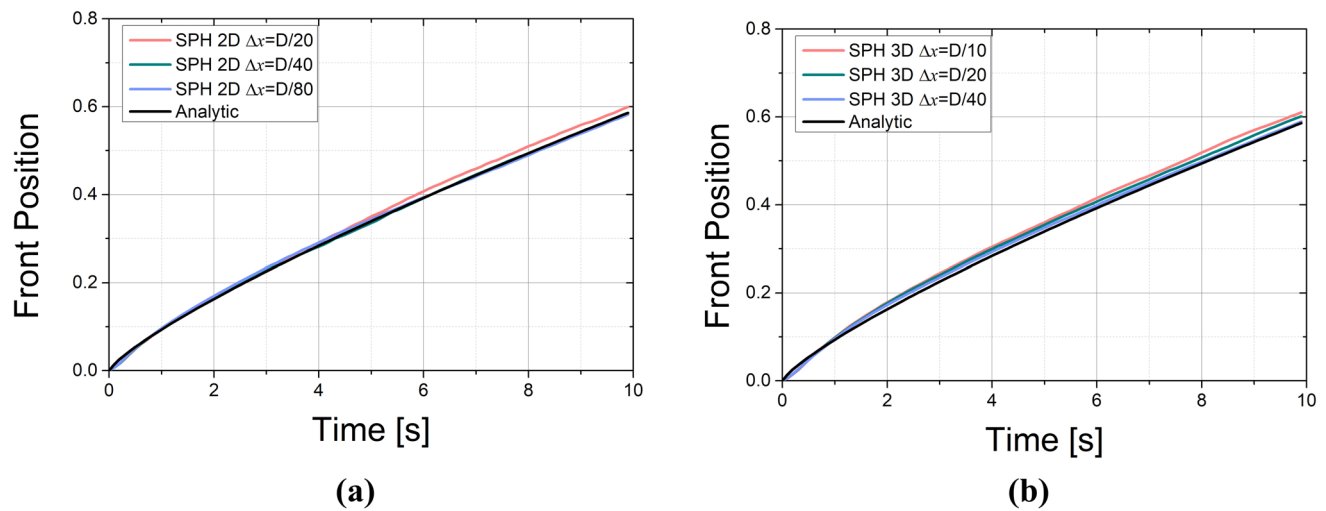
This study presents an enhanced implementation of an implicit viscosity solver within a GPU-based SPH framework for the efficient simulation of highly viscous flows. A matrix-free formulation is employed to reduce memory overhead and

facilitate parallelization, while the coefficient matrix is reformulated to preserve symmetry and strict diagonal dominance, even in the presence of complex wall boundaries. These modifications enable the effective use of iterative solvers optimized for symmetric systems, such as the conjugate gradient method, and allow robust handling of diverse boundary condition treatments.

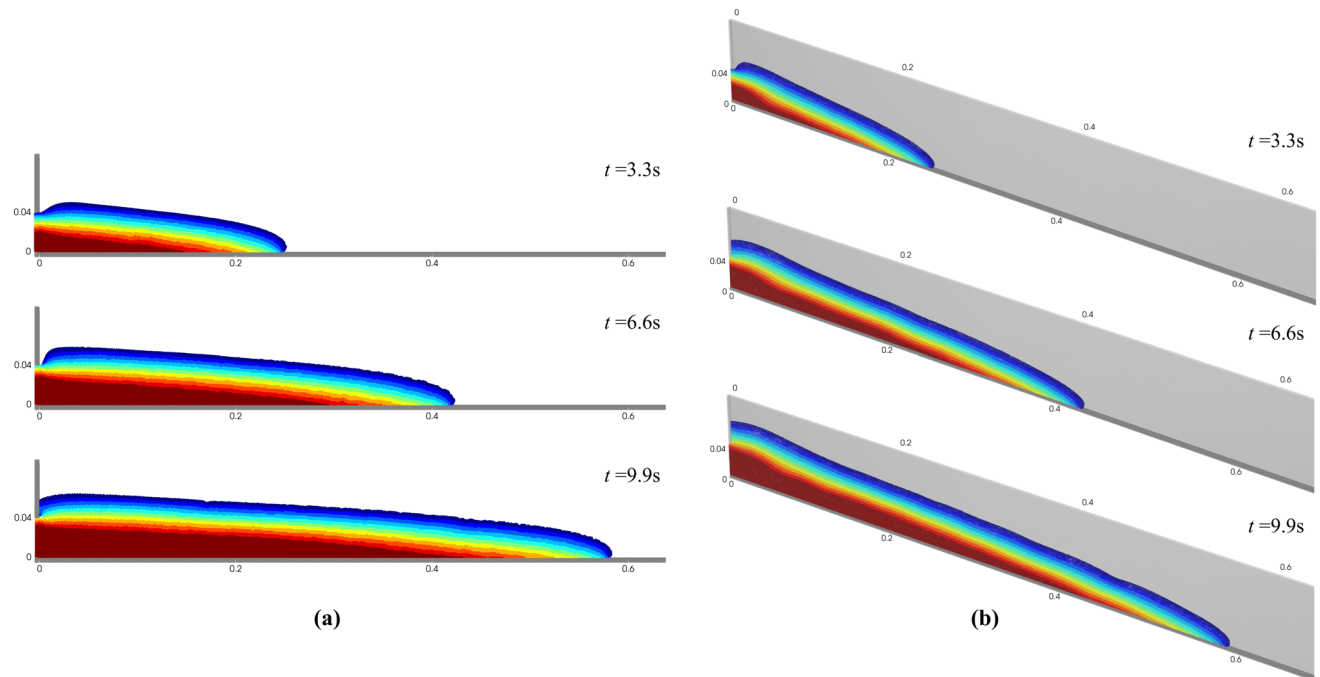
The performance of the implemented algorithm is rigorously evaluated through three benchmark cases, covering steady-state Newtonian flows (single-phase and multi-phase), a range of non-Newtonian rheologies (shear-thinning, shear-thickening, Bingham, and Herschel–Bulkley), and transient isothermal viscous spreading in both two and three dimensions. The comparative analysis confirms that the restructured matrix, constructed solely from fluid particle contributions, retains symmetry and diagonal dominance. These properties enable the effective use of Krylov methods, particularly conjugate gradient and its preconditioned form, which offer both numerical stability and computational efficiency.

In multi-phase and non-Newtonian cases, the use of preconditioning effectively mitigates the resolution-dependent growth in iteration counts, improving overall scalability. Transient simulations further confirm the solver's accuracy and stability over time, with close agreement to analytical solutions.

Overall, this study demonstrates that modest but targeted algorithmic enhancements, such as matrix reformulation and matrix-free preconditioned solvers, can yield substantial gains in the efficiency and robustness of SPH simulations involving highly viscous fluids. Future work will explore the extension of this methodology to multi-GPU environments to further improve scalability for large-scale applications.



**Fig. 24** Numerical results of isothermal spreading using PCG with different spatial resolution **a** 2D results **b** 3D results



**Fig. 25** Time evolution of isothermal spreading using PCG at fine resolution of **a** 2D results and **b** 3D results colored by pressure

**Acknowledgements** This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. 2021M2D2A1A03046881).

**Funding** Open Access funding enabled and organized by Seoul National University.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes

were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

## References

- Morikawa D, Asai M, Idris N et al (2019) Improvements in highly viscous fluid simulation using a fully implicit SPH method. *Comput Part Mech* 6:529–544. <https://doi.org/10.1007/s40571-019-00231-6>
- Chao W, Wang S, Li Y et al (2020) Natural sponge-like wood-derived aerogel for solar-assisted adsorption and recovery of high-viscous crude oil. *Chem Eng J* 400:125865. <https://doi.org/10.1016/j.cej.2020.125865>
- Zhao F, Liu Y, Lu N et al (2021) A review on upgrading and viscosity reduction of heavy oil and bitumen by underground catalytic cracking. *Energy Rep* 7:4249–4272. <https://doi.org/10.1016/j.egy.2021.06.094>
- Beloglazov I, Morenov V, Leusheva E (2021) Flow modeling of high-viscosity fluids in pipeline infrastructure of oil and gas enterprises. *Egypt J Pet* 30:43–51. <https://doi.org/10.1016/j.ejpe.2021.11.001>
- Phan DD, Horner JS, Swain ZR et al (2020) Computational fluid dynamics simulation of the melting process in the fused filament fabrication additive manufacturing technique. *Addit Manuf* 33:101161. <https://doi.org/10.1016/j.addma.2020.101161>
- Siacor FDC, Chen Q, Zhao JY et al (2021) On the additive manufacturing (3D printing) of viscoelastic materials and flow behavior: from composites to food manufacturing. *Addit Manuf* 45:102043. <https://doi.org/10.1016/j.addma.2021.102043>
- Bernasconi R, Brovelli S, Viviani P et al (2022) Piezoelectric drop-on-demand inkjet printing of high-viscosity inks. *Adv Eng Mater*. <https://doi.org/10.1002/adem.202100733>
- Jabbari M, Baran I, Mohanty S et al (2018) Multiphysics modelling of manufacturing processes: a review. *Adv Mech Eng*. <https://doi.org/10.1177/1687814018766188>
- Shao Y, Adetoro OB, Cheng K (2021) Development of multiscale multiphysics-based modelling and simulations with the application to precision machining of aerofoil structures. *Eng Comput* 38:1330–1349. <https://doi.org/10.1108/EC-10-2019-0473>
- Tan P, Zhou M, Tang C et al (2023) Multiphysics modelling of powder bed fusion for polymers. *Virtual Phys Prototyp*. <https://doi.org/10.1080/17452759.2023.2257191>
- Yoo HS, Kim ES (2024) Numerical investigation of spreading phenomena of molten corium using EISPH method. *Nucl Eng Technol*. <https://doi.org/10.1016/j.net.2024.11.003>
- Song B, Plana C, Lopez JM, Avila M (2019) Phase-field simulation of core-annular pipe flow. *Int J Multiph Flow* 117:14–24. <https://doi.org/10.1016/j.ijmultiphaseflow.2019.04.027>
- Gharibi F, Ashrafizaadeh M (2020) Simulation of high-viscosity-ratio multicomponent fluid flow using a pseudopotential model based on the nonorthogonal central-moments lattice Boltzmann method. *Phys Rev E* 101:043311. <https://doi.org/10.1103/PhysRevE.101.043311>
- Rizzieri G, Ferrara L, Cremonesi M (2024) Simulation of viscoelastic free-surface flows with the particle finite element method. *Comput Part Mech* 11:2043–2067. <https://doi.org/10.1007/s40571-024-00730-1>
- Hill BM, Mitchell TR, Łaniewski-WoŃk Ł et al (2024) Development and validation of a phase-field lattice Boltzmann method for non-Newtonian Herschel-Bulkley fluids in three dimensions. *Comput Math Appl* 176:398–414. <https://doi.org/10.1016/j.camwa.2024.10.035>
- Zago V, Bilotta G, Héroult A et al (2018) Semi-implicit 3D SPH on GPU for lava flows. *J Comput Phys* 375:854–870. <https://doi.org/10.1016/j.jcp.2018.07.060>
- Bilotta G, Zago V, Centorrino V et al (2022) A numerically robust, parallel-friendly variant of BiCGSTAB for the semi-implicit integration of the viscous term in smoothed particle hydrodynamics. *J Comput Phys* 466:111413. <https://doi.org/10.1016/j.jcp.2022.111413>
- Kondo M, Matsumoto J, Sawada T (2024) A scalable physically consistent particle method for high-viscous incompressible flows. *Comput Part Mech* 11:511–527. <https://doi.org/10.1007/s40571-023-00636-4>
- Duan G, Yamaji A, Koshizuka S (2019) A novel multiphase MPS algorithm for modeling crust formation by highly viscous fluid for simulating corium spreading. *Nucl Eng Des* 343:218–231. <https://doi.org/10.1016/j.nucengdes.2019.01.005>
- Sun X, Sakai M, Shibata K et al (2012) Numerical modeling on the discharged fluid flow from a glass melter by a Lagrangian approach. *Nucl Eng Des* 248:14–21. <https://doi.org/10.1016/j.nucengdes.2012.04.004>
- Han Y, Qiang H, Huang Q, Zhao J (2013) Improved implicit SPH method for simulating free surface flows of power law fluids. *Sci China Technol Sci* 56:2480–2490. <https://doi.org/10.1007/s11431-013-5348-5>
- Yamada D, Imatani T, Shibata K et al (2022) Application of improved multiresolution technique for the MPS method to fluid lubrication. *Comput Part Mech* 9:421–441. <https://doi.org/10.1007/s40571-021-00420-2>
- Takahashi T, Dobashi Y, Fujishiro I et al (2015) Implicit formulation for SPH-based viscous fluids. *Comput Graph Forum* 34:493–502. <https://doi.org/10.1111/cgf.12578>
- Peer A, Ihmsen M, Cornelis J, Teschner M (2015) An implicit viscosity formulation for SPH fluids. *ACM Trans Graph* 34:1–10. <https://doi.org/10.1145/2766925>
- Koschier D, Bender J, Solenthaler B, Teschner M (2022) A survey on SPH methods in computer graphics. *Comput Graph Forum* 41:737–760. <https://doi.org/10.1111/cgf.14508>
- Zhang Y, Long S, Xu Y et al (2024) Multiphase viscoelastic non-Newtonian fluid simulation. *Comput Graph Forum*. <https://doi.org/10.1111/cgf.15180>
- Lan Y, Zhang Y, Tian W et al (2023) An enhanced implicit viscosity ISPH method for simulating free-surface flow coupled with solid-liquid phase change. *J Comput Phys* 474:111809. <https://doi.org/10.1016/j.jcp.2022.111809>
- Saad Y (2003) Iterative methods for sparse linear systems. Society for Industrial and Applied Mathematics, Philadelphia
- Takahashi T, Dobashi Y, Nishita T, Lin MC (2018) An efficient hybrid incompressible SPH solver with interface handling for boundary conditions. *Comput Graph Forum* 37:313–324. <https://doi.org/10.1111/cgf.13292>
- Peer A, Gissler C, Band S, Teschner M (2018) An implicit SPH formulation for incompressible linearly elastic solids. *Comput Graph Forum* 37:135–148. <https://doi.org/10.1111/cgf.13317>
- Shimizu Y, Khayyer A, Gotoh H (2022) An SPH-based fully-Lagrangian meshfree implicit FSI solver with high-order discretization terms. *Eng Anal Bound Elem* 137:160–181. <https://doi.org/10.1016/j.enganabound.2021.10.023>
- Chen A, Erickson BA, Kozdon JE, Choi J (2024) Matrix-free SBP-SAT finite difference methods and the multigrid preconditioner on GPUs. In: Proceedings of the 38th ACM international conference on supercomputing. ACM, New York, pp 400–412
- Kiran U, Sharma D, Gautam SS (2024) An efficient framework for matrix-free SpMV computation on GPU for elastoplastic problems. *Math Comput Simul* 216:318–346. <https://doi.org/10.1016/j.matcom.2023.09.016>
- Cummins SJ, Rudman M (1999) An SPH projection method. *J Comput Phys* 152:584–607. <https://doi.org/10.1006/JCPH.1999.6246>
- Monaghan JJ (1992) Smoothed particle hydrodynamics. *Annu Rev Astron Astrophys* 30:543–574. <https://doi.org/10.1146/annurev.aa.30.090192.002551>

36. Wendland H (1995) Piecewise polynomial, positive definite and compactly supported radial functions of minimal degree. *Adv Comput Math* 4:389–396. <https://doi.org/10.1007/BF02123482>
37. Liu MB, Liu GR, Li S (2004) ?Smoothed particle hydrodynamics ? a meshfree method? *Comput Mech* 33:491–491. <https://doi.org/10.1007/s00466-004-0573-1>
38. Morikawa D, Senadheera H, Asai M (2021) Explicit incompressible smoothed particle hydrodynamics in a multi-GPU environment for large-scale simulations. *Comput Part Mech* 8:493–510. <https://doi.org/10.1007/s40571-020-00347-0>
39. Shao S, Lo EYM (2003) Incompressible SPH method for simulating Newtonian and non-Newtonian flows with a free surface. *Adv Water Resour* 26:787–800. [https://doi.org/10.1016/S0309-1708\(03\)00030-7](https://doi.org/10.1016/S0309-1708(03)00030-7)
40. Morris JP, Fox PJ, Zhu Y (1997) Modeling low Reynolds number incompressible flows using SPH. *J Comput Phys* 136:214–226. <https://doi.org/10.1006/jcph.1997.5776>
41. Hu XY, Adams NA (2007) An incompressible multi-phase SPH method. *J Comput Phys* 227:264–278. <https://doi.org/10.1016/j.jcp.2007.07.013>
42. Hu XY, Adams NA (2009) A constant-density approach for incompressible multi-phase SPH. *J Comput Phys* 228:2082–2091. <https://doi.org/10.1016/j.jcp.2008.11.027>
43. Yoo HS, Jo YB, Kim ES (2024) Comparative study of WSPH, EISPH and explicit incompressible-compressible SPH (EICSPH) for multi-phase flow with high density difference. *J Comput Phys* 506:112930. <https://doi.org/10.1016/j.jcp.2024.112930>
44. Barcarolo DA (2013) Improvement of the precision and the efficiency of the SPH method: theoretical and numerical study (Doctoral dissertation, Ecole Centrale de Nantes (ECN)).
45. Xenakis AM, Lind SJ, Stansby PK, Rogers BD (2020) An incompressible smoothed particle hydrodynamics scheme for Newtonian/non-Newtonian multiphase flows including semi-analytical solutions for two-phase inelastic Poiseuille flows. *Int J Numer Methods Fluids* 92:703–726. <https://doi.org/10.1002/fld.4802>
46. Dietemann B, Bosna F, Lorenz M et al (2020) Modeling robocasting with smoothed particle hydrodynamics: printing gap-spanning filaments. *Addit Manuf* 36:101488. <https://doi.org/10.1016/J.ADDMA.2020.101488>
47. Hosseini SM, Manzari MT, Hannani SK (2007) A fully explicit three-step SPH algorithm for simulation of non-Newtonian fluid flow. *Int J Numer Methods Heat Fluid Flow* 17:715–735. <https://doi.org/10.1108/09615530710777976>
48. Fan X-J, Tanner RI, Zheng R (2010) Smoothed particle hydrodynamics simulation of non-Newtonian moulding flow. *J Non-Newton Fluid Mech* 165:219–226. <https://doi.org/10.1016/j.jnnfm.2009.12.004>
49. Macia F, Antuono M, Gonzalez LM, Colagrossi A (2011) Theoretical analysis of the no-slip boundary condition enforcement in SPH methods. *Prog Theor Phys* 125:1091–1121. <https://doi.org/10.1143/PTP.125.1091>
50. Adami S, Hu XY, Adams NA (2012) A generalized wall boundary condition for smoothed particle hydrodynamics. *J Comput Phys* 231:7057–7075. <https://doi.org/10.1016/J.JCP.2012.05.005>
51. Pan W, Bao J, Tartakovsky AM (2014) Smoothed particle hydrodynamics continuous boundary force method for Navier-Stokes equations subject to a Robin boundary condition. *J Comput Phys* 259:242–259. <https://doi.org/10.1016/j.jcp.2013.12.014>
52. Vergnaud A, Oger G, Le Touzé D et al (2022) C-CSF: accurate, robust and efficient surface tension and contact angle models for single-phase flows using SPH. *Comput Methods Appl Mech Eng* 389:114292. <https://doi.org/10.1016/j.cma.2021.114292>
53. Green S (2010) Particle simulation using cuda. NVIDIA whitepaper 6:121–128.
54. Ahn Y, Yoo HS, Choi N, Kim ES (2024) Dynamic load balancing of multi-GPU parallelization for VULCANO VE-U7 corium spreading analysis using SOPHIA. *Nucl Technol* 211:1–21. <https://doi.org/10.1080/00295450.2024.2397617>
55. Park S-H, Jo YB, Ahn Y et al (2020) Development of multi-GPU-based smoothed particle hydrodynamics code for nuclear thermal hydraulics and safety: potential and challenges. *Front Energy Res* 8:86. <https://doi.org/10.3389/fenrg.2020.00086>
56. Tafuni A, Domínguez JM, Vacondio R, Crespo AJC (2018) A versatile algorithm for the treatment of open boundary conditions in Smoothed particle hydrodynamics GPU models. *Comput Methods Appl Mech Eng* 342:604–624. <https://doi.org/10.1016/j.cma.2018.08.004>
57. Bird RB (2002) Transport phenomena. *Appl Mech Rev* 55:R1–R4. <https://doi.org/10.1115/1.1424298>
58. Zhu H, Martys NS, Ferraris C, Kee DD (2010) A numerical study of the flow of Bingham-like fluids in two-dimensional vane and cylinder rheometers using a smoothed particle hydrodynamics (SPH) based method. *J Non-Newtonian Fluid Mech* 165:362–375. <https://doi.org/10.1016/J.JNNFM.2010.01.012>
59. Xenakis A (2016) Modelling multi-phase non-Newtonian flows using incompressible SPH. The University of Manchester (United Kingdom).
60. Foit JJ (2004) Spreading under variable viscosity and time-dependent boundary conditions: estimate of viscosity from spreading experiments. *Nucl Eng Des* 227:239–253. <https://doi.org/10.1016/J.NUCENGDES.2003.10.002>

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.